Massachusetts Institute of Technology
Lincoln Laboratory

First Test & Evaluation Report for TRACTOR C to Rust Translators

Hamed Okhravi
Noah Luther
Ian McQuoid
Brandt Ogden
Howard Mak
Nicholas Strega
Nathan Burow

October 13, 2026

DISTRIBUTION STATEMENT A. Approved for public release. Distribution is unlimited.

This material is based upon work supported by the Under Secretary of Defense for Research and Engineering under Air Force Contract No. FA8702-15-D-0001 or FA8702-25-D-B002. Any opinions, findings, conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the Under Secretary of Defense for Research and Engineering.

©2026 Massachusetts Institute of Technology.

Delivered to the U.S. Government with Unlimited Rights, as defined in DFARS Part 252.227-7013 or 7014 (Feb 2014). Notwithstanding any copyright notice, U.S. Government rights in this work are defined by DFARS 252.227-7013 or DFARS 252.227-7014 as detailed above. Use of this work other than as specifically authorized by the U.S. Government may violate any copyrights that exist in this work.

Lexington Massachusetts

This page intentionally left blank.

This page intentionally left blank.

TABLE OF CONTENTS

		Page
	List of Figures	vii
1.	EXECUTIVE SUMMARY	1
2.	INTRODUCTION	5
3.	TRANSLATOR TEAMS AND OVERVIEW OF APPROACHES	11
	3.1 Aarno Labs Translator	11
	3.2 Galois Translator	12
	3.3 Intel Corporation Translator	15
	3.4 University of Washington Translator	17
	3.5 University of Wisconsin Translator	19
	3.6 Yale University Translator	21
	3.7 Takeaways	22
4.	EVALUATION METRICS AND BENCHMARKS	31
	4.1 Evaluation Metrics and Measurement Approach	31
	4.2 Battery 01 Benchmark	41
	4.3 Milestone Projects 00 and 01 Benchmarks	42
	4.4 Evaluation Infrastructure	43
5.	EVALUATION RESULTS	45
	5.1 Functionality	45
	5.2 Safety	49
	5.3 Performance	53
	5.4 Idiomaticity	57
	5.5 Milestone Projects Evaluation	58
6.	OBSERVATIONS AND INSIGHTS	69
	6.1 Pipeline Architecture, Orchestration, and Control	69
	6.2 Correctness, Equivalence, and Evaluation Limits	69
	6.3 Safety, Undefined Behavior, and Semantic Risk	70
	6.4 Hybridization, Diversity, and System-Level Advantage	71

TABLE OF CONTENTS
(Continued)

		Page
	6.5 Source Fidelity, Configurability, and Tooling Assumptions	71
7.	CONCLUSION	73
	References	75
A	AI TERMINOLOGY GLOSSARY	79

LIST OF FIGURES

Figure No.		Page
1	Hello world examples of C-to-Rust translations	41
2	Overall Comparison for Correctness (Lower is Better)	45
3	Failure Rate by Category Across Performers	46
4	Proportional Stage of Failure by Performer	47
5	Absolute Stage of Failure by Performer	47
6	Test Failure Counts Across Performers (top 15 most failed test cases)	48
7	Kinds of Unsafe Operations Across Performers (performer-test case pairs and total calls)	49
8	Kinds of Unsafe Operations by Performer (total calls of the 110 common test cases across performers)	50
9	Unsafe Function Breakdown by Name Across Performers (performer-test case pairs and total calls)	50
10	Unsafe Operation Count per Test by Performer (variability across tests for each performer)	51
11	Comparison of Unsafe Operations vs Mean by Performer (negative means fewer unsafe operations compared to average)	52
12	Performer Summary for Unsafe Operation Use (lower average operations per test may indicate safer code generation)	52
13	Translation Runtime Distribution by Performer	54
14	Outliers of Time to Run Translation as Multiplicative Error (red means slower than expected, blue means faster)	54
15	Runtime Overhead Distribution by Performer (ratio greater than 1 means slower than C Baseline)	55
16	Memory Overhead Distribution by Performer	56
17	Types of Unsafe Operations Across Performers	61
18	Types of Unsafe Operations by Performer	61
19	Unsafe Operations per Test by Performer	62
20	Common Unsafe Functions Across Performers	62
21	Unsafe Operation Deviations by Performer	63

LIST OF FIGURES

(Continued)

Figure No.		Page
22	Translation Runtime by Performer	64
23	Runtime Overhead by Performer	65
24	Memory Overhead by Performer	65
25	Lint / Warning Count by Performer	67

1. EXECUTIVE SUMMARY

Much of today's critical software continues to rely on memory-unsafe programming languages such as C and C++. These languages underpin operating systems, networking stacks, cryptographic libraries, and safety- and mission-critical systems, yet they remain a dominant source of severe software vulnerabilities. Although their risks are well understood, decades of accumulated legacy code, performance constraints, certification requirements, and ecosystem dependencies have made large-scale rewrites impractical. Recent advances in memory-safe systems languages, program analysis, and AI-driven tooling have created a new opportunity to modernize these systems through automated or semi-automated translation rather than manual redevelopment. This shift is reinforced by recent U.S. government policy, including White House [1], NSA, and CISA [2] guidance that explicitly identifies memory safety as a foundational requirement for secure software.

The DARPA TRACTOR program seeks to develop scalable, repeatable techniques for translating large C codebases into memory-safe Rust code while preserving functionality, improving security, and maintaining competitive performance and code quality. Although fully automated translation is an aspirational goal, experience suggests that human judgment remains essential. The program therefore frames modernization as a repeatable, measurable, and security-driven process that incorporates human oversight when necessary to ensure correctness and security. MIT Lincoln Laboratory serves as the independent test and evaluation organization for the program, responsible for defining evaluation metrics, designing benchmarks, executing experiments, and analyzing results across performers.

The program comprises six performer teams pursuing diverse translation strategies (listed alphabetically): Aarno Labs and MIT; Galois, Immunant, the University of Pennsylvania, and the University of Texas at Austin; Intel and the University of California, Berkeley; the University of Washington and Better Bytes; the University of Wisconsin, the University of Illinois Urbana-Champaign, and the University of Edinburgh; and Yale University and Northeastern University. For ease of reference, we refer to the teams by their prime institutions: Aarno Labs, Galois, Intel Corporation, University of Washington, University of Wisconsin, and Yale University.

The six teams represent distinct architectural strategies for C to Rust translation (section 3). Aarno Labs's Tenjin pipeline is largely deterministic and transpiler-led, emphasizing C to C preconditioning, C2Rust-based translation [3], and staged unsafe elimination without reliance on LLM synthesis in its automated mode. Galois's CRISP system combines deterministic transpilation with bounded LLM-guided safety rewriting and provenance tracking through a multi-version intermediate representation. Intel Corporation's IDEAS system performs dependency-ordered, snippet-level LLM translation under strict no-unsafe constraints, separating internal safe Rust generation from ABI wrapper construction. University of Washington uses a tool-orchestrated, project-level LLM translation framework with an intermediate representation and explicit scheduling infrastructure, complemented by Hayroll for macro and configuration preservation. University of Wisconsin employs a whole-project, LLM-forward pipeline with structured discrepancy reporting and iterative build-driven repair. Yale University's PROCTOR focuses on static-analysis-driven Rust-to-Rust refinement after transpilation, using borrow inference and plugin-based unsafety lifting to progressively reduce unsafe constructs.

In addition, a key requirement of the TRACTOR program is that any use of LLMs must be architecturally pluggable rather than tightly coupled to a specific vendor or model release. Performer systems are therefore required to abstract model interaction behind well-defined interfaces, enabling substitution of different frontier or open-weight models without redesigning the overall pipeline. This requirement reflects both the rapid pace of model evolution and the need for long-term maintainability, cost control, and resilience against changes in model availability, performance, or policy constraints.

A central component of TRACTOR's evaluation methodology is the periodic release of standardized test batteries. Approximately every six months, MIT Lincoln Laboratory releases a battery of roughly 100 C programs that must be translated into Rust by each performer. These test cases are curated and synthesized by MIT LL to systematically exercise specific C language features, enabling controlled measurement of progress over time and direct comparison across approaches. In addition to these batteries, TRACTOR includes milestone projects that are substantially larger and more complex, intended to approximate realistic legacy software components.

This report focuses on the evaluation of the first test battery (B01) and the first two milestone projects. The first test battery was intentionally scoped to establish a baseline, focusing on well-defined pointer usage, simple data structures, conventional error handling, and static program structure, while avoiding dynamic allocation and complex aliasing. Milestone Project 0 (P00), the Perlin noise library, emphasizes numerical computation and performance-sensitive code, while Milestone Project 1 (P01), the SPHINCS cryptographic library, introduces security-critical correctness and low-level bit manipulation challenges.

Evaluation within TRACTOR is organized around four core metrics. Functional correctness is evaluated using input-output test vectors to verify that the translated Rust code faithfully matches the behavior of the original C implementations. Runtime performance is assessed by comparing execution characteristics between the translated and original code. Safety is measured by examining the translated Rust code for unsafe constructs, with the number and nature of unsafe contexts and unsafe operations serving as an indicator of remaining memory-safety risk. Translations of known memory unsafe code (i.e. undefined behavior in C) was manually evaluated for a sample set of translations for each performer. Idiomaticity is evaluated through a combination of automated Rust Clippy linting and manual review, focusing on readability, syntactic structure, and effective use of Rust's type system.

Across the 150 tests in Battery 01, performer success rates varied substantially. The highest overall functional correctness rate was achieved by Yale University at 98.7 percent (148 of 150 tests correct), followed by Aarno Labs at 95.3 percent (143 of 150). The overall average failure rate across performers was 17.8 percent. Detailed comparative correctness results are presented in section 5.1, including failure distributions by origin and visibility and stage of failure. Performers also scored differently in the other evaluation dimensions. Intel Corporation achieved the best safety score of 1.56 unsafe operations per test case while for runtime performance University of Wisconsin generated the fastest Rust code (median 4% overhead). For idiomaticity, Intel Corporation has the best overall idiomatic Rust code with readability, syntax, and types scores of 2.67, 2.89, and 2.67 (all out of 3). The evaluation of Milestone Projects 0 and 1 (P00 and P01) indicate similar trends. Aarno Labs, Intel Corporation, University of Wisconsin, and Yale University achieved the highest functionality scores of 100% (40 of 40) while University of Washington, University of Wisconsin, and Intel Corporation scored the highest on idiomaticity with an average scores of 4.2, 3.6, and 3.4 out of 5 respectively.

Results from this initial evaluation have already yielded several significant technical insights into the current state of automated C-to-Rust translation. Most notably, static analysis-based approaches tend to achieve higher functional correctness, whereas LLM-based approaches produce more idiomatic Rust code. In other words, static analysis techniques are generally more successful at generating code that behaves correctly, but the resulting code is often less aligned with Rust best practices and conventions (they are more C-like). By contrast, LLM-based methods generate code that is more natural and idiomatic from a Rust perspective, but less consistently passes functionality tests. This contrast suggests that hybrid approaches combining the strengths of both techniques may ultimately offer the most effective overall solution.

Across evaluated systems, effective LLM-based translation pipelines are converging on a common, software-like control structure, with architectural performance driven more by orchestration choices than by model selection. Trade-offs center on translation granularity, where hybrid approaches balance global coherence with localized error detection. Practical correctness is bounded by test-based equivalence rather than more formal notions of equivalence, with realistic and withheld tests exposing late-stage semantic mismatches. Correctness, safety, and idiomaticity remain largely independent objectives, and preserving explicit errors or undefined behavior is often safer than silent semantic patching. *No single approach dominates across all cases*; instead, complementary strengths and isolated failures suggest that ensemble or multi-stage pipelines can outperform monolithic designs, while tooling and environment assumptions remain a nontrivial source of build instability. These findings highlight both encouraging progress and ongoing limitations, and they help identify key challenges and opportunities that will shape future research directions within the TRACTOR program.

This page intentionally left blank.

2. INTRODUCTION

Grace Hopper famously warned that "the most dangerous phrase in the language is: 'We've always done it this way.' " Nowhere is this observation more relevant than in the construction of critical software.

For decades, many of the world's most important software systems have been implemented in memory-unsafe languages such as C and C++ [4]. These languages underpin operating systems, communication stacks, cryptographic libraries, embedded controllers, and a wide range of safety- and mission-critical applications. Their performance characteristics and low-level control made them compelling historical choices. Yet their weaknesses are equally well understood [5]. Undefined behavior, memory corruption, buffer overflows, and use-after-free errors remain among the most common and most damaging sources of software vulnerabilities [6]. Despite this persistent risk, vast legacy codebases, performance sensitivities, certification constraints, and ecosystem dependencies have rendered wholesale rewrites largely impractical. As a result, many high-risk systems remain anchored to decades-old implementation decisions.

The urgency of addressing memory safety is not new [7]. For more than twenty years, researchers, security practitioners, and industry leaders have repeatedly identified memory-unsafe languages as a systemic root cause of exploitable vulnerabilities [8]. Academic studies, breach analyses, and exploit campaigns have consistently traced failures back to memory corruption and undefined behavior [9]. What is unusual about the present moment is not recognition of the problem, but the breadth of agreement about its implications. Technical agencies such as the NSA and CISA [2], national policy bodies [1], and even consumer advocacy organizations [10] have converged on a similar conclusion: incremental mitigation is insufficient, and structural change at the language level is required [11].

The software ecosystem has also explored large-scale language modernization before. Efforts such as IBM's Cobol-to-Java modernization work more than two decades ago [12] demonstrated the feasibility of large-scale translation, but they operated in a very different technical and policy environment. Those efforts focused primarily on platform modernization and maintainability rather than on eliminating systemic security vulnerabilities at their root.

Indeed, over the intervening decades, the community invested heavily in attempting to compensate for memory-unsafe languages without abandoning them. Compiler-based mitigations such as stack canaries, address space layout randomization, and control-flow integrity [13–15] were introduced to detect or constrain exploitation. Dynamic defenses such as sandboxing and runtime instrumentation were deployed to limit damage [16]. Static analysis tools, verification techniques [17], hardened allocators [18], and secure coding standards such as MISRA and CERT were developed to reduce such vulnerabilities [19, 20]. Each of these advances improved resilience in specific contexts. Yet none eliminated the underlying classes of vulnerabilities. Memory corruption continues to appear across browsers, operating systems, network services, and embedded platforms [21]. In this sense, the shift toward memory-safe languages is not a first attempt at remediation, but the culmination of repeated partial solutions that have not fundamentally resolved the problem [22].

In recent years, however, a qualitatively different path has emerged. Advances in safe programming languages, language tooling, and artificial intelligence have opened the possibility of automated or semi-automated translation of legacy code into safer languages with strong correctness and security guarantees, most notably Rust [23–30]. Rather than rewriting systems from scratch, these approaches aim to mechanically

transform existing implementations while preserving behavior and strengthening safety properties. This line of work draws on static and dynamic program analysis, formal methods, program synthesis, and increasingly large language models. Together, these techniques offer a path to scale beyond what manual refactoring or isolated rewrites can realistically achieve.

Rust [31] occupies a uniquely attractive niche within this modernization landscape. Unlike managed languages that depend on garbage collection, Rust enforces memory safety through compile-time ownership and borrowing rules without imposing runtime GC overhead, preserving deterministic performance characteristics required in systems and real-time domains. It interoperates directly with C and C++ through stable foreign function interfaces, enabling incremental migration rather than all-or-nothing rewrites. It delivers performance comparable to C and C++, in part because it leverages the same LLVM compiler infrastructure and provides explicit control over memory layout and representation. These properties make Rust not merely a safe language for new development, but a technically viable and strategically compelling target for large-scale automated translation.

This broader shift has increasingly been reflected in national policy. In 2024, the White House Office of the National Cyber Director issued Back to the Building Blocks: A Path Toward Secure and Measurable Software, explicitly encouraging the adoption of memory-safe languages and reduction of reliance on inherently memory-unsafe ones [1]. In 2025, the NSA and CISA jointly released Memory Safe Languages: Reducing Vulnerabilities in Modern Software Development, emphasizing the role of language-level safety in improving software resilience and outlining practical pathways for adoption [2]. These statements build upon years of technical consensus and frame memory safety as a national security priority rather than merely a best practice [22].

Against this backdrop, the DARPA TRACTOR program [32] represents a concerted research effort to fundamentally change how legacy, memory-unsafe software is modernized. TRACTOR seeks to develop scalable, automated techniques for translating large C codebases into memory-safe Rust. Although fully automated translation is an aspirational goal, experience suggests that human judgment remains essential. The program therefore frames modernization as a repeatable, measurable, and security-driven process that incorporates human oversight when necessary to ensure correctness and security. Its goals extend beyond functional correctness to include strengthened safety guarantees, competitive performance, and code quality suitable for long-term maintenance and evolution. In doing so, TRACTOR directly addresses the longstanding technical challenge—and growing policy imperative—of reducing systemic dependence on memory-unsafe software.

MIT Lincoln Laboratory serves as the independent test and evaluation organization for the TRACTOR program. In this capacity, MIT LL is responsible for designing, curating, and executing a rigorous evaluation framework that allows DARPA to objectively assess progress across the program. This role includes defining evaluation metrics, developing standardized benchmarks, running experiments across performers, and analyzing results to surface technical insights and limitations.

The program includes six performer teams that pursue various translation strategies (listed alphabetically): Aarno Labs and MIT; Galois, Immunant, the University of Pennsylvania, and the University of Texas at Austin; Intel and the University of California, Berkeley; the University of Washington and Better Bytes; the University of Wisconsin, the University of Illinois Urbana-Champaign, and the University of Edinburgh; and Yale University and Northeastern University. For brevity, we refer to these teams by their prime institu-

tions: Aarno Labs, Galois, Intel Corporation, University of Washington, University of Wisconsin, and Yale University.

These six teams represent distinct architectural approaches to C to Rust translation (section 3). Aarno Labs's Tenjin pipeline is largely deterministic and transpiler-centric, emphasizing C to C preconditioning, C2Rust-based translation [3], and staged removal of unsafe code, without depending on LLM synthesis in its automated mode. C2Rust [3] is a rule-based C-to-Rust transpiler developed prior to the TRACTOR program. It produces Rust code that is semantically equivalent to the original C, but the generated output typically contains substantial unsafe code and unidiomatic constructs, requiring significant refinement to achieve fully safe and idiomatic Rust. Galois's CRISP system combines deterministic transpilation with bounded LLM-guided safety rewriting and provenance tracking through a multi-version intermediate representation. Intel Corporation's IDEAS system performs dependency-ordered, snippet-level translation using LLMs under strict no-unsafe constraints, explicitly separating internal safe Rust generation from ABI wrapper construction. University of Washington employs a tool-orchestrated, project-level LLM translation framework centered on an intermediate representation and explicit scheduling infrastructure, augmented by Hayroll to preserve macros and configuration artifacts. University of Wisconsin uses a whole-project, LLM-forward pipeline with structured discrepancy reporting and iterative, build-driven repair. Yale University's PROCTOR focuses on static analysis driven Rust-to-Rust refinement after transpilation, leveraging borrow inference and plugin-based unsafety lifting to progressively reduce unsafe constructs.

A key element of the TRACTOR evaluation methodology is the periodic release of standardized test batteries. Approximately every six months, MIT Lincoln Laboratory releases a new battery consisting of around 100 C test cases that must be translated into Rust by each performer. These test cases are both curated and synthesized by MIT LL and are designed to systematically exercise specific features of the C language. Each battery is intentionally scoped to isolate particular technical challenges, allowing evaluators to track progress over time and identify strengths and weaknesses in different translation approaches.

The first TRACTOR test battery[1] was deliberately scoped to exercise a constrained but representative subset of the C language, establishing a baseline for translation quality while minimizing complexity. It focuses on pointer usage with well-defined lifetimes, including pointers to stack- or statically allocated data, with no use of `malloc` or `void*`, and permits pointer arithmetic only in controlled forms such as indexing into arrays or parsing `argv` using `argc`. The battery includes pass-by-reference patterns for values and `structs` that do not involve pointer arithmetic, testing the ability to map C's implicit reference semantics to Rust's explicit borrowing model. Error-handling idioms are exercised through conventional return codes and interactions with `errno` via `errno.h`. In addition, the battery emphasizes global and static program structure, including global state, compile-time constants, static variables, and static initialization, all of which are pervasive in legacy C code. Data structures are intentionally simple, consisting largely of plain `structs` without pointers and with sizes known at compile time, allowing the evaluation to focus on correctness, safety, and idiomatic representation without introducing dynamic memory or complex aliasing.

Evaluation within TRACTOR is conducted along four primary dimensions. Functional correctness is measured using input-output test vectors to determine whether the translated Rust code faithfully reproduces the behavior of the original C implementation. Runtime performance is evaluated by comparing the execution characteristics of the translated code against the original. Safety is assessed by analyzing the translated Rust

[1] Publicly released at https://github.com/DARPA-TRACTOR-Program/PUBLIC-Test-Corpus.

code for unsafe operations, using the number and nature of unsafe blocks as a proxy for residual memory-safety risk. In addition, translations of known memory unsafe code (i.e. undefined behavior in C) was manually evaluated for a sample set of translations for each performer. Finally, idiomaticity is evaluated using a combination of automated Rust Clippy linting and manual scoring that assesses readability, syntactic structure, and appropriate use of Rust's type system.

Across the 150 tests in Battery 01, performer success rates varied significantly, revealing meaningful differences in robustness under the evaluation framework. The highest overall functional correctness rate was achieved by Yale University, which successfully translated 148 of 150 tests (98.7 percent) with Aarno Labs second at 95.3 percent (143 of 150). Across all performers, the average failure rate was 17.8 percent. A detailed breakdown of correctness outcomes, including failure distributions by test origin, visibility, and stage of failure, is presented in section 5.1. Notably, most failures across teams occurred at the semantic comparison stage rather than during translation or compilation, underscoring that build success alone is not a sufficient proxy for correctness.

Performance also diverged across the other evaluation dimensions. In safety, measured by the mean number of unsafe operations per test case, Intel Corporation achieved the strongest result at 1.56 unsafe operations per test, followed closely by Galois and Yale University (section 5.2). For runtime overhead relative to the original C baseline, all teams remained within a modest range, but University of Wisconsin produced the fastest Rust artifacts with a median overhead of approximately 4 percent (section 5.3). In idiomaticity, based on manual scoring of readability, syntax, and type usage (each on a 1 to 3 scale), Intel Corporation achieved the highest overall scores with averages of 2.67, 2.89, and 2.67 respectively, indicating consistently idiomatic Rust output. Together, these results demonstrate that no single performer dominates across all dimensions, and that improvements in one metric often correspond to tradeoffs in another.

The evaluation of Milestone Projects 0 and 1 (P00 and P01) indicate similar trends. Aarno Labs, Intel Corporation, University of Wisconsin, and Yale University achieved the highest functionality scores of 100% (40 of 40). Conversely, University of Washington, University of Wisconsin, and Intel Corporation scored the highest on idiomaticity with an average scores of 4.2, 3.6, and 3.4 out of 5 respectively.

This initial evaluation of the TRACTOR performers has already surfaced a number of important technical insights regarding the current state of automated C-to-Rust translation. These insights point to both promising progress and persistent challenges that will shape future research directions in the program. A high-level list of these insights include:

- **Static Analysis and LLM Approaches Exhibit a Correctness–Idiomaticity Tradeoff.** Initial evaluation results reveal a clear tradeoff between correctness and idiomaticity in automated C-to-Rust translation. Static analysis-based approaches more reliably generate functionally correct code, but the output often remains structurally C-like and less idiomatic in Rust. In contrast, LLM-based approaches produce more natural, Rust-idiomatic code, but with lower functional reliability. These findings suggest that hybrid approaches integrating static analysis with LLM-based generation may provide the most effective end-to-end translation solution.
- **Convergence of LLM-based translation pipelines.** Despite surface-level differences in prompting style and model choice, effective LLM-based translators converge on a shared control structure: scoped context, strict behavioral constraints, explicit feedback signals, and machine-ingestible out-

put contracts. Prompts increasingly function as engineered software artifacts rather than ad hoc instructions.

- **Granularity and orchestration dominate architectural trade-offs.** Whole-project translation improves global coherence but increases context pressure and obscures error provenance, while granular or dependency-ordered translation localizes failures at the cost of more complex orchestration. Evaluation results indicate that hybrid strategies combining these approaches are already justified.

- **Test-based equivalence defines the practical boundary of correctness.** Successful compilation is common across systems and insufficient as a correctness signal; most failures arise from late-stage semantic mismatches detectable only through test-based equivalence. Organic and hidden tests expose weaknesses not visible in synthetic or public cases, underscoring the importance of realistic and withheld workloads.

- **Correctness, safety, and idiomaticity remain decoupled objectives.** High functional correctness does not imply reduced unsafety or improved idiomaticity, and vice versa. On safety, performers often chose to preserve unsafe behavior, e.g., a buffer overflow, to exactly match C semantics, while others instead opt for runtime errors. A third approach is to modify the behavior of the application, e.g., by dropping an input that would trigger a buffer overflow. This third approach is trades the risk of an incorrect and hard to diagnose change in application behavior against the potential reward of patching the unsafe behavior.

- **Different approaches yield complementary strengths.** No test case was failed by all performers, and most failures are isolated to a single translator, indicating that different systems handle different semantic corner cases well. This strongly suggests that ensemble or multi-stage pipelines can outperform any single monolithic approach.

- **Performance variability is driven more by orchestration than models.** Translation runtime varies significantly across and within performers, with outliers primarily attributable to orchestration decisions such as iteration depth and feedback loops rather than underlying model capability. Pipeline control logic is therefore a first-order performance concern.

- **Tooling and environment assumptions affect build outcomes.** Some build failures stem from dependency packaging and environment mismatches between performer containers and standalone evaluation contexts. In particular, reliance on unreleased or container-local dependencies highlights the need for more portable and reproducible artifact generation.

This page intentionally left blank.

3. TRANSLATOR TEAMS AND OVERVIEW OF APPROACHES

This section provides an overview of each performer's approach to translation. It is not intended to be a comprehensive technical treatment; detailed descriptions are deferred to publications from the individual performer teams. Rather, the goal is to summarize how each team frames and addresses the problem, highlighting key commonalities and differences among their respective approaches. Viewed together, these approaches reveal several recurring architectural patterns and design tradeoffs.

Throughout this report, we extensively use AI-related terminology. A brief description of these terms are provide in Appendix A. Readers should refer to AI-related literature for a more extensive treatment of the subject.

Across the six performers, three recurring pipeline families emerge: deterministic transpile-and-refine systems that start from C2Rust and progressively eliminate unsafe (Aarno Labs, Yale University), LLM-led project-scale translators that rely on build and repair loops to converge (University of Wisconsin, University of Washington), and hybrid approaches that transpile first and then apply bounded LLM-guided safety and repair passes (Galois), with Intel Corporation occupying a distinct niche through dependency-ordered, snippet-level LLM translation under strict safety gates. The primary architectural axis is granularity: whole-project prompting maximizes global coherence but increases context pressure and regression risk, whereas incremental or analysis-heavy pipelines localize failures at the cost of greater orchestration complexity. Compilation success is universally treated as a baseline, but test-based equivalence is the practical differentiator between "it builds" and "it behaves the same," making test coverage and test generation central to confidence. Unsafety reduction strategies vary from outright prohibition (Intel Corporation), to iterative elimination loops (Aarno Labs, Galois), to boundary isolation and static-analysis-driven lifting (Yale University), suggesting complementary rather than competing techniques. Macros and conditional compilation remain a shared challenge, with University of Washington's Hayroll standing out as a reusable asset for preserving configurability. Finally, prompting itself functions as an engineered control surface rather than ad hoc instructions: performers systematically constrain model behavior through structured context, explicit goals and invariants, tightly specified output contracts, and iterative feedback injection, pointing toward a broader pattern of treating prompts as programmable, testable components within the translation pipeline.

3.1 AARNO LABS TRANSLATOR

Aarno Labs and MIT's approach centers on **Tenjin**, a staged translation pipeline that treats "safe, correct, and idiomatic Rust" as the output of successive refinement rather than a single conversion step. Tenjin uses **C2Rust**[2] [3] as a core building block for baseline C→Rust transpilation, but wraps it with substantial **pre-translation refactoring (C→C)** and **post-translation improvement (Rust→Rust)** passes. In its current fully automated mode, Tenjin is **non-LLM**: it prioritizes predictable transformations, repeatable builds, and incremental cleanup validated primarily through compile/check cycles driven by the project's build configuration (e.g., compilation databases derived from build commands).

[2] c2Rust is a rule-based C-to-Rust transpiler developed prior to the TRACTOR program. It produces Rust code that is semantically equivalent to the original C, but the generated output contains substantial unsafe code and unidiomatic constructs, requiring significant refinement to achieve fully safe and idiomatic Rust. In many cases, c2Rust effectively wraps all its generated Rust code in an unsafe block. c2Rust was developed by Immunant, Inc who is part of the Galois team but is also supporting all performers.

A defining feature is Tenjin's C→C **"preparatory refactoring"** stage, which reshapes C into forms that translate more cleanly into safe Rust. Using build-system information (compile commands, extracted defines), Tenjin applies refactorings such as **macro (non) expansion / preprocessor refolding**, **declaration splitting** (e.g., breaking multi-declarations into simpler units), and **global-state localization**. Global localization is especially central: Tenjin identifies mutable globals (and supporting type/definition closure), constructs a struct holding that state, instantiates a singleton in `main`, and threads a pointer through the call graph so uses become explicit field accesses rather than ambient mutation. Tenjin also performs refactorings like **lifting subfield arguments** (e.g., `foo(x, x->y)` → `tmp = x->y; foo(x, tmp)`), which reduces multi-aliasing patterns that later create borrow/ownership pressure in Rust.

For C→Rust translation, Tenjin runs **an upstream c2rust transpilation** and also a **Tenjin-modified c2rust transpilation** as part of its pipeline, treating c2rust as a reusable engine rather than a complete solution. Tenjin supports an explicit **guidance** mechanism (a JSON schema) to steer ambiguous choices—for example, selecting target Rust types for specific variables/fields, forcing or preventing `mut`, marking plain old data (POD) types to enable safe casting strategies (e.g., via `bytemuck`), selecting substitute crates for known libraries, and asserting constraints like "no `math` errno usage." Guidance can be produced by humans or LLMs, but Tenjin's current automated workflow uses neither directly.

Tenjin then runs a Rust→Rust **improvement pipeline** that treats the initial translation as a starting point: standard cleanup (`cargo fmt`, `cargo fix`, clippy-based fixes), dead-code trimming, and normalization such as removing unnecessary numeric casts and stripping c2rust-introduced lint allowances. A major focus is **iterative unsafe elimination**: Tenjin extracts a call graph (via rustc APIs), processes functions in dependency-aware/topological order, and repeatedly applies "remove unsafe → re-check" steps (typically `cargo check`) to ensure the code remains buildable as `unsafe` is reduced. Validation, as reflected in the pipeline description, is therefore primarily **compile/check-driven** rather than test-driven (tests may still be applied externally, but are not the core loop emphasized here). Tenjin's distinguishing tradeoff is that it is **highly auditable and stable** (deterministic transforms + explicit guidance), but reaching strongly idiomatic designs in difficult codebases may require more explicit guidance than LLM-forward approaches that infer intent opportunistically.

Guidance interface (Aarno Labs's Tenjin): Tenjin's main "steering surface" is a JSON guidance object (`-guidance`) that can be supplied as a literal or file path. Table 1 summarizes the guidance keys and intent.

3.2 GALOIS TRANSLATOR

Galois, Immunant, the University of Pennsylvania, and the University of Texas at Austin's approach is embodied in **CRISP** ("C to Rust for Idiomatic and Safe Programming"), which combines a deterministic orchestration pipeline with targeted LLM-driven rewrites. A central organizing abstraction is **MVIR** ("Multi-Version Intermediate Representation"), which records each transformation (transpilation attempts, rewrites, repairs) as a node in a provenance graph. This framing treats translation as a traceable sequence of candidates and edits rather than a single one-shot conversion, supporting auditability and allowing the pipeline to prefer the most suitable intermediate version for subsequent steps.

TABLE 1

Aarno Labs's Tenjin guidance schema (summary of available keys and their intent).

Guidance key	Purpose
`vars_of_type`	Map serialized Rust types to variable specifiers that should use that type (supports wildcards like `*:name`).
`declspecs_of_type`	Like `vars_of_type`, but applies to global declarations.
`vars_mut`	Map variable specifiers to booleans indicating whether the corresponding Rust declarations should be `mut`.
`fn_return_type`	Map function names to the Rust return type they should be made to / assumed to return.
`pod_types`	List of type names treated as "plain old data" (enables safe casting strategies, e.g., `bytemuck`).
`using_crates`	Specify third-party crates to prefer during translation (restricted to an allow-list).
`no_math_errno`	Assert that translated code does not use `errno` via math library functions (debug/testing constraint).

CRISP begins with a robust **initial transpilation** stage designed to handle real C projects (including conditional compilation). It attempts a translation using **Hayroll** (University of Washington's c2rust wrapper that aims to recover conditional compilation structure in Rust), and falls back to a conventional **c2rust** transpilation if the Hayroll output fails to compile. Build and test actions are executed in a containerized/sandboxed environment (Docker), emphasizing repeatability and constraining side effects during iterative repair and validation.

A distinctive architectural choice is **FFI function splitting**: exported entry points that must preserve ABI are rewritten into (1) a small exported wrapper (often necessarily unsafe due to raw-pointer/ABI constraints) and (2) an internal implementation function that CRISP then tries to make safe. This isolates unavoidable unsafety at the boundary while enabling deeper parts of the program to become safe Rust. After splitting, CRISP runs bounded iterative loops that (a) measure remaining unsafety, (b) invoke an LLM "safety rewrite" to eliminate unsafe operations and remove `unsafe` blocks when possible, and then (c) run nested repair loops that use compiler and test feedback to fix compilation errors and behavioral discrepancies. In addition to passing raw logs, CRISP can *inline* structured compiler errors into source as comments prior to an LLM repair step, which acts as a prompt-shaping technique to localize edits.

CRISP makes its evaluation posture explicit. **Unsafety** is measured structurally by walking the Rust syntax tree to identify unsafe functions and unsafe blocks, while ignoring unsafety inside exported/FFI entry points where it is often unavoidable. **Correctness** is judged using any available tests using pass/fail outcomes as the primary correctness signal. In this evaluation no tests were generated and thus successful compilation was the only pass/fail outcome used. The LLM component is used as a bounded rewrite engine within this deterministic loop; in the current container configuration, CRISP defaults to **OpenAI GPT-5**. The main tradeoffs are the usual ones: test-driven equivalence can miss untested behaviors, and LLM rewrites can introduce subtle semantic drift if constraints are insufficiently tight. MVIR provenance, sandboxed

TABLE 2

Galois's CRISP prompt roles and contracts as observed in containerized workflow.

Prompt	When used	Key constraints	Inputs supplied
`llm_safety`	Unsafe-reduction step	Preserve behavior; replace unsafe operations with safe equivalents; remove `unsafe` blocks; **FFI entry points (e.g., `#[no_mangle]`) must keep signatures unchanged**	Selected Rust files (with file paths); sometimes additional context/instructions
`llm_repair_compile`	After `cargo check` fails	Fix compile errors; avoid introducing new `unsafe` unless absolutely necessary; prefer safe replacements	Rust files + compiler stderr/logs
`llm_repair`	After tests fail (or full build/test step)	Fix to compile and pass tests; avoid adding new `unsafe` beyond what already exists	Rust files + build/test logs
`inline_errors` (shaping step)	Before repair	Inject structured diagnostics as inline comments to focus edits	Diagnostic JSON/logs + prior Rust source

execution, and incremental validation mitigate these risks by preserving intermediate versions and checking each iteration's build/test outcomes.

LLM interface and prompting (Galois's CRISP): CRISP's LLM usage is organized around a small set of prompt templates that correspond to pipeline roles (unsafe removal, compile repair, test repair), with strict output contracts to enable deterministic parsing and file overwrite.

Prompting conventions observed in Galois's CRISP

- **Style:** conversational English; direct tone; sometimes first-person framing ("I tried compiling...").
- **Formatting:** lightly structured with Markdown-like conventions; strict output contracts (code in fenced blocks, file-path headers) to support automated extraction/overwrite.
- **Core phrases:** "convert to safe Rust," "don't change its behavior," "replace unsafe operations," "remove unsafe blocks," and "FFI entry point signature must not change."
- **Rationale cues:** prompts may include short "do X so that you may Y" purpose statements to steer the model toward acceptable transformations.
- **Operational note:** overly long file paths can confuse weaker models (observed as a prompting consideration); occasional role-setting/roleplay instructions appear in some templates.

3.3 INTEL CORPORATION TRANSLATOR

Intel Corporation and University of California, Berkeley's approach is organized around **IDEAS** ("Improved Decoding and Equivalence Automated testing at Scale"), which treats translation as an incremental, dependency-ordered synthesis problem rather than a whole-project rewrite. The pipeline builds a symbol-/dependency view of the input C code (derived from compilation database information and extracted AST structure), then translates *one C definition at a time* into Rust in topological order so that earlier translations can serve as stable "reference" context for later ones. Instead of asking an LLM to invent an entire crate at once, IDEAS repeatedly integrates each new translation into a Rust workspace/harness and uses `cargo build` feedback to steer bounded retries until the program compiles.

Correctness is operationalized through tight integration with a build-and-test harness. Each translated unit is checked for compatibility with existing Rust "reference_code" and a Rustified view of already-translated dependencies; compiler diagnostics become immediate feedback to the translation loop. IDEAS also emphasizes faithful low-level semantics—notably, its translation guidance treats integer arithmetic as *wrapping* by default and instructs the model to implement bitwise operations exactly as written (especially shifts/rotations), reflecting an intent to avoid common semantic drift in C→Rust ports where overflow and bit-twiddling behavior differs. Test-driven equivalence is part of the stated design intent (randomized input generation and differential checks), but the most concrete current capability described in the implementation notes is translation/adaptation of provided test vectors into a cargo-test-consumable format rather than a fully realized autonomous test generator loop.

IDEAS draws a sharp line between **safe, idiomatic Rust** for internal logic and **C-ABI compatibility** at the boundary. The core translator (`ideas.translate`) is engineered to produce Rust that contains *no unsafe constructs*; this is enforced mechanically by compiling translations under a "no-unsafe" regime (e.g., `RUSTFLAGS="-D unsafe-code"`), so that any unsafe introduced by the model fails the build immediately. This pushes the system toward safe signatures (e.g., converting pointer+length pairs into slices), explicit ownership choices, and idiomatic APIs where possible. Conversely, C ABI decoration, symbol exporting, and unsafe boundary manipulations are delegated to a separate wrapper stage (`ideas.wrapper`), which can generate C-compatible wrappers (via bindgen scaffolding plus LLM completion) that explicitly convert between wrapper-layout types and the crate's internal types.

The system's distinctive strengths are (1) **granularity** (symbol/snippet-level translation with dependency ordering), (2) **hard safety guardrails** ("unsafe" is not merely discouraged, it is rejected by the build), and (3) **prompt engineering as an optimizable artifact**. IDEAS uses DSPy [33] to structure prompts, extract code reliably from model outputs, log trajectories, and (in principle) optimize prompts against user-defined metrics such as "compiles cleanly." DSPy is a declarative, programmatic approach to building and optimizing language model pipelines that abstracts prompt engineering into structured program components instead of manual string writing, enabling more systematic and reproducible optimization. IDEAS also introduces a wrapper-generation prompt distinct from the snippet-translation prompt, reflecting an architectural separation between internal Rust quality and external ABI constraints. Tradeoffs follow from the same choices: dependency extraction is necessarily imperfect in the general case (particularly with complex build systems and macro-driven structure), and the tight "no helpers / no refactors / no extra items" constraints can make certain idiomatic restructurings harder to realize when deeper redesign would be beneficial.

TABLE 3

Intel Corporation's IDEAS prompting roles and contracts as observed in containerized workflow.

Prompt / signature	When used	Key constraints	Inputs supplied
Snippet→Rust translation	Per C definition (dependency order)	**No unsafe allowed**; integrate with provided Rust harness; do not redeclare reference/dependent items; preserve identifiers; no `extern "C"` / no `#[no_mangle]`; prefer slices over raw pointers; **wrapping semantics** for integer ops; implement bitwise ops exactly; avoid new helpers/wrappers/extra globals	`reference_code` (Rust), `snippet` (C), `dependent_code` (C), plus `prior_translation` and `cargo build` feedback
FFI wrapper completion	After bindgen scaffold generation	Only modify specified wrapper body; convert between `crate::wrapper::*` and internal crate types by copying; call `crate::symbol`; convert outputs back; wrapper may contain unsafe; use build feedback to iterate	`example_wrapper` (Rust), target symbol name, crate path, prior wrapper, and `cargo build` feedback

LLM interface and prompting (Intel Corporation's IDEAS) Intel Corporation's IDEAS uses multiple prompt "signatures" aligned to pipeline roles. The two most central are (1) snippet-level translation into safe Rust, and (2) C-FFI wrapper completion (filling in bindgen-generated `unimplemented!()` bodies with correct conversions and calls). Prompt construction and response handling are mediated by DSPy (templating, code extraction, logging) and may be optimized against build-based metrics; the prompt-optimization component is described as using a Genetic-Pareto style optimizer (GEPA [34]), which maintains a set of prompts that are iteratively improved based on a reward function in an attempt to find the globally optimal prompt.

In the current container configuration, model routing is observed as: **OpenRouter** with a primary model of **GPT-5**.

Prompting and control-loop conventions observed in Intel Corporation's IDEAS

- **Granularity:** translate one C definition at a time in dependency order; previously translated Rust becomes structured context (`reference_code`).
- **Hard safety gate:** the translation stage is compiled under "no unsafe" rules; unsafe is treated as a build-breaking violation, not merely a lint.
- **Semantic guardrails:** default-to-wrapping arithmetic and "implement bitwise exactly" instructions reduce common C→Rust drift for low-level code.
- **Separation of concerns:** internal Rust translation is kept free of ABI/export decoration; C-ABI compatibility is handled in a separate wrapper module.
- **Optimization posture:** prompts are treated as structured programs (DSPy) and may be tuned against a build-based metric such as "compiles cleanly." [34]

3.4 UNIVERSITY OF WASHINGTON TRANSLATOR

University of Washington and Better Bytes' approach is organized around a tool-orchestrated translation framework named **HARVEST** (often described in the repo as **HARVEST-Translate**) that is designed to run translation as an iterative process over an internal intermediate state. Conceptually, HARVEST frames C→Rust conversion as a loop that can integrate multiple capabilities—"lifting" (extracting structure/specs/intent), analysis (static and dynamic), machine-learning assistance, "lowering" (generating Rust), and validation—with each step recorded for diagnostics and repeatability. In its current implementation emphasis, the system is structured as a *scheduler + tool runner + IR storage* pipeline: tools update a shared IR, the scheduler selects the next tools to execute, and diagnostics capture why each step ran and what it produced.

In the present toolset, HARVEST-Translate focuses on an end-to-end **project-level translation** workflow: it loads the raw C sources, determines whether the target is a library or executable, and then invokes an LLM-driven translator that emits a Rust project layout (including `Cargo.toml`) and the translated Rust sources. A `cargo build` step is then used as a primary feasibility check, returning either build artifacts or aggregated compiler diagnostics. The architectural intent is extensibility: additional tools can be introduced to perform deeper lifting, targeted rewrites, or validation and then feed their results back into subsequent steps. As implemented today, the most visible "loop" is the repeated execution of tools over the evolving IR rather than a deeply layered, multi-stage repair pipeline.

Alongside the translation framework, HARVEST contributes a major enabling technology for real-world C codebases: **Hayroll**, a c2rust wrapper aimed at preserving the semantics and structure of **macros and conditional compilation**. A core limitation of preprocessing-based translation is that only one configuration of compile-time flags is translated, so alternate code paths and configurability can disappear in the resulting Rust. Hayroll addresses this by (i) analyzing preprocessor conditions to characterize when lines are included, (ii) running the preprocessor across multiple macro-definition settings, (iii) translating each resulting variant with c2rust, and (iv) merging the variants into a single Rust implementation guarded by Rust `#cfg` conditions. Hayroll also supports reconstructing macro expansions into more readable Rust constructs

TABLE 4

University of Washington's HARVEST-Translate LLM prompting modes and contracts as observed in containerized workflow

Prompt / mode	When used	Key constraints	Inputs supplied
Project→Rust (`cargo lib`)	Library targets	Translate full project; preserve external interfaces; prefer canonical safe Rust internally; **do not translate comments / do not add new comments**; emit Rust files + `Cargo.toml`	JSON array of C/CMake/etc. files (path + contents); "return as JSON" output contract
Project→Rust (`cargo bin`)	Executable targets	Same as library mode, with stricter formatting: **valid JSON only** (no Markdown fences), extreme care for JSON validity; emit Rust entry point + `Cargo.toml`	JSON array of project files (path + contents); explicit instruction to output JSON only

(e.g., `macro_rules!` or functions), using program analyses (including inputs derived from macro-usage analysis work such as **Maki** [35]) to guide which regions can be safely lifted into reusable Rust abstractions.

The defining tradeoff of HARVEST's current "project-in, project-out" translation posture is **automation and throughput** versus **control over fine-grained semantic edge cases**. Whole-project prompting can rapidly produce a coherent Rust crate structure, but it increases context burden and can make it harder to isolate which specific code element caused a downstream mismatch. HARVEST's design mitigates this by placing translation inside a tool-driven loop with explicit diagnostics and a validation step, and by providing Hayroll as a complementary mechanism for preserving configurability that would otherwise be lost in a preprocessing-centric translation. In a cross-performer comparison, HARVEST is best viewed as (1) an extensible orchestration framework for integrating translation/analysis/validation tools, and (2) the source of a macro/conditional-compilation recovery layer (Hayroll) that other pipelines can reuse.

LLM interface and prompting (University of Washington's HARVEST-Translate): HARVEST-Translate's LLM interaction is framed as "translate a C project into a Rust project," where the entire input project is supplied as a JSON list of files (path + contents), and the model is required to emit a structured JSON response representing the Rust project files. In the current container configuration, HARVEST routes translation via **OpenRouter** and uses **x-ai/grok-code-fast-1** as a primary model, with options to use **Ollama** and **OpenAI gpt-4o**.

Prompting conventions observed in University of Washington's HARVEST-Translate

- **Project-level context:** inputs are provided as a whole-project file bundle (JSON list of `path`/`contents`), encouraging globally consistent crate structure.
- **Strict output contract:** responses must be machine-ingestible JSON representing output files; executable mode emphasizes "JSON only" (no preface, no code fences).
- **Interface discipline:** preserve external interfaces while preferring safe, idiomatic Rust internally; avoid comment translation and avoid adding new comments.
- **Validation hook:** `cargo build` is used to determine success and to aggregate compiler diagnostics for downstream use in the tool loop.

3.5 UNIVERSITY OF WISCONSIN TRANSLATOR

The University of Wisconsin and University of Illinois at Urbana-Champaign's current TRACTOR implementation is best characterized as an **LLM-forward, whole-project translation workflow** with structured refinement loops. The system first gathers build context (notably `compile_commands.json`) and aggregates the project's C sources, then prompts an LLM to translate the *entire project* into a Rust target that builds with Cargo. In the current container configuration, the translation is performed using **Claude Sonnet 4.5 via Google Cloud**. A distinctive operational choice is that the translated Rust is constrained to live in a **single destination Rust file**, with the rest of the Rust project structure (e.g., `Cargo.toml`) generated around it.

The system's stated fidelity target is strict: prompts repeatedly emphasize **exact equivalence** (matching observable behavior), **no simplifications**, and preserving **API/ABI expectations** (especially for libraries exposed through an `extern "C"` interface). Instead of relying on a single translation pass, the implementation runs multiple **post-translation refinement steps** that explicitly interrogate mismatches and build failures. After initial translation, it performs a **per-C-file semantic check** that asks the LLM to locate likely behavioral discrepancies between a specific C file and the monolithic Rust translation, using cross-file declaration summaries as additional context. It then runs a follow-on LLM step to **apply targeted semantic fixes** to the Rust file using only the discrepancy report for that C file as guidance, aiming to improve functional alignment without destabilizing other translated components.

Once semantic alignment passes are applied, the system shifts to a **build-cleanup loop** driven by `cargo build` diagnostics. Build issues are handled in priority order: it attempts to eliminate **compiler errors** first, and only once errors are resolved does it attempt to reduce **warnings**. These repair prompts repeat the same equivalence constraints (no simplifications; preserve behavior; preserve ABI), while adding pragmatic guardrails that reflect common C→Rust drift points: align integer signedness and widths with the C code; preserve loop bounds and indexing behavior; avoid out-of-range indexing; and remove or correctly use unused items rather than blanketing the code with `#[allow]` attributes unless needed for API/ABI compatibility. The system caps this process (e.g., a bounded number of iterations) to prevent open-ended refinement.

TABLE 5

University of Wisconsin prompting roles and contracts as observed in containerized workflow

Prompt / stage	When used	Key constraints	Inputs supplied
Whole-project translation	Initial step	Exact equivalence; **no simplifications**; build with Cargo; preserve CLI/stdin/stdout (exec); preserve extern C API/ABI (lib); prefer safe+idiomatic Rust; stable Rust; stdlib-only unless necessary; **single Rust file output**	`compile_commands` blob + concatenated `c_sources` blob + destination path
Per-file discrepancy report	After initial translation (per C file)	Report only (no code); identify missing functionality/control-flow/type/indexing/global-state mismatches for the specific C file; output "NO SIGNIFICANT ISSUES FOUND" when applicable	One C file + other-file declaration summaries + full translated Rust source
Semantic fix for one C file	After discrepancy report (per C file)	Apply fixes described in discrepancy report; preserve prior correct behaviors; **no simplifications**; output full Rust file (single destination)	Discrepancy report + focus C file + other-file declarations + current Rust file
Build-driven repair (errors / warnings)	After `cargo build`	Fix errors first, then warnings; preserve equivalence and ABI; align integer types; avoid indexing drift; remove dead code when possible; output full Rust file	Build output + current Rust + full C sources

In published research directions relevant to this performer lineage, multi-step methodologies resemble two complementary themes: (1) **incremental decomposition** into translation units with validation at each step (including dynamic-analysis-mined properties such as pointer aliasing/nullability and unit-level equivalence testing), and (2) **two-stage translation** where an initial semantics-preserving (often unidiomatic) translation is followed by an idiomatic refinement step guided by analysis signals and validated through end-to-end testing [36, 37]. The current TRACTOR implementation described above is more monolithic than these decomposition-first research designs, but it approximates the same spirit through its per-file discrepancy analysis and bounded refinement loops: it aims to retain whole-program context for coherence while still introducing intermediate validation hooks (discrepancy reports and build diagnostics) to constrain and correct LLM behavior.

LLM interface and prompting (University of Wisconsin): The prompting strategy is organized as a small set of role-specific templates: (1) whole-project translation, (2) per-file discrepancy detection (report-only), (3) semantic fix application (code output), and (4) build-driven repair (code output), with strict output contracts (e.g., "return only the full Rust source in a "`rust` fenced block") so the pipeline can reliably extract and overwrite the destination file.

Prompting conventions observed in University of Wisconsin workflow

- **Fidelity language repeats:** "exactly equivalent," "matches observable behavior," and **"do not make any simplifications"** appear across stages to reduce drift between translation and repair.
- **Whole-project context first:** the initial translation prompt includes both `compile_commands` and the full C source blob to encourage a globally consistent Rust crate.
- **Single-file constraint:** all generated Rust is consolidated into one destination file, simplifying extraction but concentrating edit risk.
- **Refinement via structured checks:** discrepancy reports are report-only (not code), then used as bounded input to targeted semantic repair prompts.
- **Build cleanup posture:** eliminate errors before warnings; discourage blanket `#[allow]` and prefer removing truly dead/unused code when safe to do so.

3.6 YALE UNIVERSITY TRANSLATOR

The Yale University and Northeastern University's approach is embodied in **PROCTOR** ("Compositional Certified Translation of Concurrent C to Idiomatic Rust"), whose current container emphasizes **static-analysis-driven Rust-to-Rust transformation** rather than LLM-based synthesis. The pipeline begins by transpiling C into Rust using a **fork of c2rust**, producing a baseline translation that is often structurally faithful and can compile when provided the correct build context, but still contains significant `unsafe` due to raw pointers, mutable globals, unions, and unsafe library calls. PROCTOR then applies a sequence of automated rewrite passes (implemented as plugins) to progressively convert this baseline into **safer and more idiomatic Rust**, while tracking quantitative unsafety metrics throughout.

A central technical component is PROCTOR's **pointer and borrow inference** strategy. The system attempts to retype raw pointers into safe Rust references where feasible using a pipeline of analyses (ownership/aliasing-related reasoning, mutability, and "fatness" distinctions between array-like pointers and scalar pointers). Its borrow inference is described as an **iterative demotion** process: it begins with an optimistic assumption that pointers can be safe references, checks those assumptions against Rust-like borrowing constraints, and then selectively demotes only the pointers involved in violations (e.g., back to raw pointers or other less-strong types), iterating until reaching a fixed point. This approach aims to maximize safe-reference coverage while remaining conservative when C patterns cannot be represented safely without stronger invariants.

Beyond pointer retyping, PROCTOR uses a **plugin-based unsafety lifting pipeline** (via **Crat**) to address common sources of unsafety and non-idiomatic structure. These plugins include (1) replacing certain `extern` usages with idiomatic Rust imports, (2) straightforward preprocessing/syntax rewrites to clean up c2rust artifacts, (3) pointer-focused conversions to references/slices where supported by analysis, (4–5) I/O and libc normalization (e.g., mapping common libc calls to stdlib equivalents when behavior permits), and (6) elimination of **mutable statics** by rewriting global state into safer containers such as `Cell`/`RefCell` (and using thread-local storage where appropriate). In addition, PROCTOR integrates **Crown** as a complementary Rust-to-Rust transformation that can retype and rewrite raw pointers according

TABLE 6

Yale University's PROCTOR post-c2rust unsafety-lifting pipeline as a sequence of Crat plugin stages.

Plugin stage	Purpose (as implemented/claimed in the tool pipeline)
Extern	Replace certain `extern` usage patterns with idiomatic Rust imports (`use`)
Preprocessing	Syntax cleanup / normalization of translation artifacts (e.g., "fresh" temporaries)
Pointer analysis	Retype raw pointers to references/slices using borrow inference + fatness + mutability reasoning
I/O	Replace common libc I/O idioms with stdlib counterparts where behavior permits
Libc	Broader libc→stdlib substitutions where supported
Static	Eliminate `static mut` via `Cell`/`RefCell` or thread-local storage when needed

to inferred ownership and usage properties, further reducing internal `unsafe` (while acknowledging that some boundary and low-level behaviors may remain inherently unsafe) [38].

PROCTOR's validation and optimization posture is largely **tool- and metric-driven**. The runner exposes modes to transpile, evaluate idiomaticity, evaluate unsafety, execute translated code on available tests, run Crown, run clippy, iterate clippy fixes to a fixed point, and perform explicit "unsafety lifting" passes. Unsafety is quantified using feature-oriented counts (e.g., unsafe pointers and unsafe pointer uses; mutable-static usage; unsafe calls; union field access), giving the pipeline a concrete objective beyond "compile successfully." The principal tradeoff is that static analysis must be conservative in the face of ambiguous C aliasing and lifetime behavior; PROCTOR may therefore retain unsafety where proofs or strong invariants are unavailable. In return, the approach is highly auditable and reproducible: it does not depend on stochastic model outputs and instead improves translations through explicit analyses, deterministic rewrites, and measurable reductions in unsafe constructs.

Borrow inference by iterative demotion (conceptual)

- Start optimistic: treat pointers as safe references when possible.
- Check against Rust-like borrowing constraints on the translated program.
- Demote only the pointers implicated in violations (becoming more conservative types).
- Iterate until reaching a fixed point with no remaining violations under the current typing.

3.7 TAKEAWAYS

This section distills high-level insights from the six performer approaches, focusing on recurring design patterns, tradeoffs, and opportunities for cross-pollination of tools and techniques.

3.7.1 Three recurring pipeline families

Across performers, translation pipelines cluster into three broad families (with hybrids in practice):

1. **Transpiler-led, deterministic refinement:** start from c2rust (or a fork), then invest heavily in deterministic refactoring and Rust-to-Rust improvement to reduce `unsafe` and improve idiomaticity (Aarno Labs; Yale University).
2. **LLM-led, project-scale translation with repair loops:** translate large project context directly with an LLM, then validate and (in some systems) iteratively refine using diagnostics (University of Wisconsin; University of Washington).
3. **Hybrid, transpile first then LLM-guided safety/repair:** use deterministic translation to "get to Rust," then apply LLM transforms in bounded loops to reduce `unsafe` and fix compile/test outcomes (Galois).

Intel Corporation (IDEAS) is a hybrid of a different kind: it is *LLM-driven* but not project-monolithic. Instead, it performs **snippet/symbol-level** translation under tight build constraints and safety gates, with a separate wrapper stage for ABI concerns.

3.7.2 Granularity and context management is a primary design axis

One of the clearest differentiators is **what unit of code is translated/rewritten at a time**:

- **Whole-project prompting (University of Wisconsin; University of Washington):** maximizes global context and can yield coherent crate structure quickly, but increases context burden and makes it harder to localize regressions. It tends to rely on additional mechanisms (e.g., build loops; University of Wisconsin also uses discrepancy reports).
- **Whole-codebase transforms but file-scoped rewrites (Galois):** still supplies substantial context to the LLM, but uses bounded loops and provenance tracking (MVIR) plus targeted repair steps to make changes more attributable.
- **Dependency-ordered incremental translation (Intel Corporation):** reduces context size per step and makes failures local to a single definition, but requires extracting a usable dependency graph and maintaining a stable harness/reference context.
- **Deterministic refactor + transpile + improve (Aarno Labs; Yale University):** shifts difficulty into analysis and rewrite engineering rather than prompt/context management.

NOTE (context pressure): several systems effectively "send everything at once" to the model (whole-project). This magnifies the value of slicing/RAG-like context selection or dependency-driven prompting (the directionIntel Corporation and research systems like Syzygy pursue [36]).

3.7.3 Correctness: compiles-cleanly is common; test-equivalence is the differentiator

All approaches treat **successful builds** as a minimum bar, but they diverge on what constitutes correctness beyond compilation:

- **Compile/check-centric validation:** Aarno Labs's improvement loop and Intel Corporation's snippet loop both rely heavily on compile feedback as the tightest iteration signal; Intel Corporation further uses a hard ban on unsafe in the translation stage.
- **Test-driven correctness:** Galois's correctness metric is explicitly tied to test pass/fail outcomes (typically the test vectors provided by the test and evaluation team). University of Wisconsin uses build diagnostics and also prompts the model to identify behavioral discrepancies per C file before applying fixes.
- **Research-direction equivalence:** Approaches such as Syzygy [36], a recent academic work also exploring the translation of C to safe Rust using LLMs and automated test generation, emphasize incremental translation and incremental equivalence checking and dynamic property mining to constrain signature and ownership choices.

A key takeaway is that **test-based equivalence is the practical separator** between "it compiles" and "it behaves the same," but its value depends on test coverage and the ability to generate or broaden tests.

3.7.4 Safety and `unsafe` reduction: four distinct strategies

Performers attack unsafety using qualitatively different mechanisms:

1. **Hard prohibition of unsafe (Intel Corporation):** unsafe is not merely discouraged; it is made build-breaking for translation outputs. This enforces safe signatures (e.g., slices) and pushes boundary-unsafe work into a separate wrapper stage.
2. **Iterative unsafe elimination loops (Aarno Labs, Galois):** start from unsafe Rust and systematically remove unsafe operations with repeated check/repair cycles. Aarno Labs does this deterministically; Galois uses LLM rewrites guided by an unsafety metric and bounded repair loops.
3. **Boundary isolation patterns (Galois and Intel Corporation):** isolate unavoidable unsafety to the ABI boundary via wrapper splitting (Galois) or separate wrapper modules (Intel Corporation).
4. **Static-analysis-driven lifting (Yale University):** use analyses and plugin pipelines to retype pointers, eliminate `static mut`, and replace libc/I/O patterns with safer idioms without relying on LLM synthesis.

These are complementary rather than mutually exclusive: a performer could combine (e.g.) pointer retyping (like Yale University) with hard unsafe gating (like Intel Corporation) for certain translation stages.

3.7.5 Macros and conditional compilation are a shared pain point; University of Washington's Hayroll is a reusable asset

The C preprocessor is a textual transformation stage that runs before compilation, expanding macros and conditionally including or excluding code based on compile-time flags (e.g., `#define`, `#ifdef`, `#if`). A recurring limitation of preprocessing-centric approaches is **loss of configurability**: only one macro configuration gets translated, and alternate code paths disappear. University of Washington's Hayroll

directly addresses this by multi-configuration preprocessing, translation, and merge into Rust `#cfg`-guarded code, and by reconstructing macro expansions into readable Rust abstractions. This is one of the few places where a concrete, reusable tool exists and is already leveraged cross-performer (Galois using University of Washington's Hayroll).

This suggests a natural portability pathway:

- Transpiler-led systems (Aarno Labs and Yale University) could incorporate Hayroll when configurability preservation matters.
- LLM-forward systems (University of Wisconsin) could use Hayroll output as a higher-quality, structure-preserving "C baseline" or as additional context to reduce macro-related drift.

3.7.6 Prompting as an engineered interface (not just "instructions")

Across LLM-using performers, prompts function as a *software interface* between an orchestrator and a code generator/rewriter. The most useful way to compare prompting is not by model choice, but by how each team constrains model degrees of freedom along four axes: **Context** (what is shown), **Goals/constraints** (what must hold), **Data/feedback** (what evidence is injected), and **Formatting** (how outputs are made machine-ingestible).

A shared backbone: Context → Goals → Data → Formatting. Despite surface differences in prose and tone, the major prompt templates converge on a repeatable skeleton:

1. **Context/setting:** what the artifact is (translated-from-C Rust; a C project; a single snippet; an FFI wrapper).
2. **Goal:** equivalence and/or safety ("convert to safe Rust," "exactly equivalent," "compiles cleanly").
3. **Constraints:** what cannot change (FFI signatures; identifiers; no refactors; no helpers; stdlib-only).
4. **Data:** the concrete inputs plus the iteration signal (files/snippet + build logs/test logs/discrepancy reports).
5. **Output contract:** JSON-only, fenced code blocks, or item-only Rust output suitable for direct compilation.

The main differentiators are *how much* context is provided at once and how strictly the orchestrator forces the model into a narrow edit lane.

Four prompt families that map to performer architectures.

- **Role-templated rewrites with conversational "I tried X" framing (Galois):** Galois's prompts are short, direct, and often first-person (e.g., "I tried compiling... I got an error"). They are split by *role*

(`llm_safety`, `llm_repair_compile`, `llm_repair`) with consistent constraints and a consistent overwrite-friendly format (file path + Markdown code block). This style is optimized for iterative repair: the model is repeatedly asked to take one bounded corrective step conditioned on logs.

- **Spec-by-demonstration with JSON I/O (University of Washington):** University of Washington uses "you are a code translation tool" prompts plus an explicit worked example showing the *exact JSON schema* expected. This is less about natural language nuance and more about schema compliance: "return as JSON" (and for executables, "valid JSON only; never any preceding markdown"). The example functions as both a behavioral demonstration and an output validator.

- **Sectioned templates with tagged input blocks and intermediate report artifacts (University of Wisconsin):** University of Wisconsin's prompts are strongly structured: a header (target name/-type/destination), bullet requirements (notably repeated "no simplifications" / "exactly equivalent" language), and explicitly delimited data blocks like <`compile_commands`> and <`c_sources`>. The distinctive piece is the *two-step semantic refinement*: first a **report-only discrepancy prompt** (no code allowed), then a fix prompt that consumes that report. This inserts a reviewable intermediate artifact that can reduce unconstrained rewrites.

- **Prompt-as-code signatures with metric-aligned iteration (Intel Corporation):** Intel Corporation is the most explicit about treating prompts as programmable artifacts. DSPy "signatures" define typed fields (e.g., `reference_code`, `snippet`, `dependent_code`, `prior_translation`, `feedback`) and enforce consistent extraction/logging. Two separate signatures encode separation of concerns: snippet translation (safe Rust, no unsafe) and wrapper completion (fill in `unimplemented!()` in a bindgen scaffold with explicit conversion rules). Prompt optimization against user-selected metrics is treated as an engineering goal (GEPA-style optimizer [34]).

Constraints are not generic; each team encodes a different "semantic risk model." The constraint language reveals what each performer expects to go wrong:

- **Unsafety removal vs. unsafety prohibition:** Galois repeatedly says "remove unsafe blocks" and "replace unsafe operations," while Intel Corporation prohibits unsafe outright in translation outputs (forcing safe signatures and pushing boundary-unsafe work to wrappers).

- **Behavior drift under overflow/bit-twiddling:** Intel Corporation explicitly mandates wrapping integer semantics and "bitwise exactly as written" to avoid a known class of C→Rust semantic mismatches.

- **Drift under "simplification":** University of Wisconsin repeats "do not make any simplifications" across translation and repair, and also enumerates common mismatch classes (loop bounds, indexing, integer signedness, array/string handling, global state).

- **Boundary invariants:** Galois requires unchanged signatures for `#[no_mangle]` / `#[export_name]` entry points; Intel Corporation forbids ABI decoration in the translation stage and handles ABI separately.

Prompt hygiene is operational, not cosmetic. Several prompt systems encode "plumbing" lessons as implicit requirements for stability: strict output contracts, stable formatting, and bounded edit scopes. Galois explicitly notes an ergonomics issue (overly long file paths can confuse weaker models), and all systems depend on practical decisions like log truncation/ordering and how diagnostics are presented.

Non-LLM analogue: structured guidance as a control surface (Aarno Labs). Even without LLMs, Aarno Labs's `-guidance` JSON plays a similar role to an LLM prompt contract: it externalizes ambiguity-resolution (types, mutability, POD markings, crate choices) into a machine-readable interface that can be human-, tool-, or LLM-produced.

Portable lessons for other performers. Two prompt-engineering techniques appear broadly reusable:

- **Insert intermediate artifacts to bound edits:** University of Wisconsin's report-only discrepancy stage and Galois's error-inlining both create localized, reviewable guidance that reduces "freeform rewriting."
- **Treat prompts as software:** Intel Corporation's DSPy signatures and metric-aligned optimization posture suggest a general pattern: prompts should be versioned, testable, and tunable against explicit pipeline metrics rather than treated as static prose [34].

3.7.7 Cross-pollination opportunities (tools/ideas likely to transfer)

Several techniques appear directly portable across performer architectures:

- **Aarno Labs's C→C refactors as preconditioning for LLM translation:** global localization and subfield-argument lifting reduce aliasing/borrow pressure and could improve downstream LLM translation stability (especially for whole-project translators).
- **Intel Corporation's prompt engineering:** using prompt-as-code modules such as DSPy with prompt optimizers can avoid human bias in prompt generation and lead to more effective LLM interactions.
- **Intel Corporation's and Galois's "no-unsafe" wrappers:** separation of safe and necessarily-unsafe rust code allows fixed-point and direct unsafety removal tools to converge more quickly on a safe translation.
- **Galois's provenance tracking (MVIR) for LLM-heavy pipelines:** recording transformation variants as first-class artifacts makes debugging, rollback, and attribution easier in stochastic rewrite loops.
- **University of Wisconsin's discrepancy reports as guardrails:** report-only discrepancy identification before code changes is a useful intermediate validation artifact that could be adopted by project-scale translators (including HARVEST-Translate).
- **Yale University's plugin pipelines for predictable idiomaticity lifts:** deterministic libc/I/O normalization and `static mut` elimination can be valuable post-passes even for LLM-produced code.

TABLE 7

Prompt interfaces across LLM-using performers (as reflected in notes and container-observed workflows).

Performer	Prompt style / structure	Output contract	Feedback injection
Galois	Conversational, role-templated (safety / compile repair / test repair); lightly Markdown-structured; explicit FFI signature rule	File path + fenced Rust code blocks	Compiler logs and/or test logs; optional diagnostic inlining into source
Intel Corporation	DSPy signatures (typed fields) for snippet translation and wrapper completion; explicit "no extra items" constraints	Rust items only (translation); constrained wrapper-body edits (wrapper stage)	Per-iteration `cargo build` feedback; prompt-as-code templating/logging; metric-aligned tuning [34]
University of Washington	"Translation tool" framing; spec-by-demonstration with explicit JSON input/output examples	Strict JSON-only file bundle	`cargo build` success/failure and aggregated diagnostics
University of Wisconsin	Sectioned template with header + bullet requirements; tagged blocks (<compile_commands>, <c_sources>); separate report-only stage	Full destination Rust file in "'rust fence	Discrepancy reports (report-only) → semantic fix; then build errors/warnings loops

- **University of Washington's Hayroll as a shared macro/configurability layer:** macro recovery is sufficiently specialized that reusing Hayroll is likely cheaper and less error-prone than re-implementing it independently.

Table 8 illustrates a compact comparison of the performers' approaches in using LLMs.

TABLE 8

High-level comparison of backbone and translation granularity across performers.

Performer	Backbone	Granularity
Aarno Labs	transpile+refine	project + passes
Galois	transpile+LLM	project/files
Intel Corporation	LLM + harness	symbol/snippet
University of Washington	LLM + orchestration	whole project
University of Wisconsin	LLM + refinement	whole project + per-file checks
Yale University	transpile+analysis	project + plugins

This page intentionally left blank.

4. EVALUATION METRICS AND BENCHMARKS

4.1 EVALUATION METRICS AND MEASUREMENT APPROACH

The metrics described here are intended to capture the key dimensions relevant to evaluating a C-to-Rust translation. These dimensions include functional correctness, safety, performance, and idiomaticity, listed in approximate order of importance.

4.1.1 Functional Correctness

A Rust translation is *functionally correct* if it produces the same observable input/output behavior as the original C program on all test vectors: matching standard output, standard error, return code, and—for library targets—the expected output state exposed through the C ABI. This is an *empirical*, test-based notion of correctness, not a formal one; we make no claim of semantic equivalence in the logical or proof-theoretic sense, and correctness is bounded by the coverage of the test suite.

Undefined behavior (UB) in C complicates this definition. The C standard permits a compiler to do anything when UB is encountered, so there is no single "correct" output to match. Nevertheless, for the purposes of this evaluation we treat correctness as *behavioral fidelity to the compiled C binary*: if the C code, as compiled by `clang`/LLVM on the evaluation platform, deterministically produces a specific output for a given input—even when that output arises from UB—we expect the Rust translation to produce that same output. In other words, *C means what LLVM does*: the reference behavior is not the abstract C standard but the concrete execution produced by the LLVM toolchain on the test system. A translation that silently changes the observable output is considered incorrect for the purposes of this metric; conversely, a translation that preserves the UB-derived output is considered correct. Test vectors that exercise known UB are currently excluded from automated scoring (`has_ub` flag), and their handling is assessed separately and informally; improved automated characterization of UB is planned for future evaluation cycles.

To evaluate functional correctness, we cultivated *test cases*—or C programs/projects collectively referred to as our *test corpus*—in two ways. The first was isolating smaller parts of larger *organic* projects to evaluate translators on a real-world C code. We then manually (either by hand or with the help of a Large Language Model (LLM)) generated *synthetic* projects to test certain features that could not be easily found or replicated in real-world programs.

An important distinction between test cases is the split between those which generate executables vs. a shared library (`.so` file). Test cases meant to produce a shared library are marked with `_lib` at the end of their name to signify this.

The split between executable and library test cases is important because each lends itself to a certain style of translation. Executable test cases allow for heavier optimizations, allowing them to refactor function signatures and the structure of the project, with the only requirement to maintain the input/output (I/O) behavior present in the original C executable. A library test is slightly different as it requires the preservation of symbols and signatures present in the original C library. This makes optimizations more difficult, requires the use of `unsafe`, but allows the Rust library to be a drop-in replace of the C library—an important property for a gradual translation from C to Rust.

To accompany each test case we generated (either by hand or with the help of a LLM) *test vectors* that represent the I/O behavior that both the C and Rust projects must adhere to. This necessitated the creation of a specialized JavaScript Object Notation (JSON) schema designed to capture everything involved with a running executable or library. Table 9 outlines each field in the schema, along with its purpose (is it used for executables, libraries, or both). `stdout` and `stderr` allow for more permissive matching using regular expressions if specified (e.g., floating point comparisons).

TABLE 9

Test vector schema

Field	Description	Purpose
`argv`	command-line arguments	executable
`stdin`	input provided to program	executable/library
`stdout`	expected output from program	executable/library
`stderr`	expected output to standard error	executable/library
`rc`	expected return code	executable
`lib_state_in`	provided input state (see **Cando**)	library
`lib_state_out`	expected output state (see **Cando**)	library
`has_ub`	description of undefined behavior	executable/library

To facilitate automated functional equivalence testing we developed two tools, Continuous Integration (CI)/Rust runner scripts[3], and `cando`. Both of these tools consume the specification from Table 9 to check if a C or Rust program conforms to it. Our CI scripting arose from the need for a sanity check for the test vectors we generated for the original C. Therefore, we can be certain all the generated test vectors exactly match the runtime behavior of their corresponding C test case. Our Rust runner scripts are similar, and currently tightly coupled to our CI, but are designed to run translated Rust projects from performers and our baselines. They consume the same JSON specification, and ensure translated Rust exhibits the same runtime behavior as the original C (to the extent that the test vectors cover all possible program states). These scripts act as a higher-level orchestrator for running functional equivalence, and other evaluation metrics across the test corpus. The current implementation skips all test vectors marked with `has_ub`, though better automatic characterization of UB will be a future improvement.

[3] Publicly released at https://github.com/DARPA-TRACTOR-Program/PUBLIC-Test-Corpus/tree/main/deployment/scripts/github-actions.

Cando `cando`[4] is a tool developed by us (in Rust) to assist with determining functional equivalence at the individual test case level. It was primarily designed to support library tests, but in the future will be expanded to encompass binary tests as well (which is currently implemented by the CI/Rust runner scripting). `cando` uses the same specification defined in Table 9 for the fields marked as used for library tests. Notably, `lib_state_in` and `lib_state_out` are used to represent the *state* to pass into the library, and expected out state of the library respectively. This can include things like function arguments, return values, environment variables, file names, and file contents. These are managed and passed to the corresponding shared library (a `.so` file generated either by C or Rust) using a manually generated *harness* for each test case. The harness specifies where to locate the shared library, along with what *symbol* (function) to call.

4.1.2 Safety

Safety is a complimentary goal to functionality in that neither is complete without the other. While it is important for a C-to-Rust translation to be functionally equivalent to the original C program, it is also important for that translation to add additional security guarantees that are typical from a native Rust program. We evaluate Rust translations for safety in two ways, the use of the `unsafe` keyword, and undefined behavior (UB).

Unsafety Usefully analyzing and quantifying the use of unsafe constructs within Rust code is a subtle question. For this first report, we use two approaches to evaluate the use of unsafe constructs: one syntactic[5] and the other semantic[6]. We intend to continue refining these tools in future batteries to encourage performers to minimize the use of unsafe constructs where at all possible.

Our syntactic analysis parses the Rust source code and analyses the Abstract Syntax Tree (AST) to report on the use of constructs that indicate unsafe code contexts. This measures the quantity of unsafe blocks, unsafe trait implementations, unsafe function declarations, statements within unsafe contexts, and miscellaneous uses of the `unsafe` keyword. In the case of unsafe function declarations, private functions are distinguished from public functions, as often the use of an unsafe function at a library boundary is required for successful interoperation with C code. As a result, performers should not be penalized for the mere presence of unsafe function declarations. However, skilled translations should be able to minimize the scope of unsafety within these required unsafe entry points.

Our semantic analysis instruments the Rust compiler to produce a report on the use of any operation that *requires* an unsafe context, as opposed to a statement that just happens to be in an `unsafe` block. This includes calls to unsafe functions, uses of inline assembly, uses of mutable statics, raw pointer dereferences, accesses of union fields, and more.

One notable complication is worth mentioning here: in the Rust 2024 edition[7], the semantics of unsafe function declarations were altered such that their bodies are no longer implicitly considered unsafe contexts.

[4] Publicly released at https://github.com/DARPA-TRACTOR-Program/PUBLIC-Test-Corpus/tree/main/tools/cando2.

[5] Publicly released at https://github.com/DARPA-TRACTOR-Program/PUBLIC-Test-Corpus/tree/main/tools/rust_eval/static/measure_unsafety.

[6] Publicly released at https://github.com/DARPA-TRACTOR-Program/PUBLIC-Test-Corpus/tree/main/tools/rust_eval/unsafe_ops.

[7] See *The Rust Edition Guide*: https://doc.rust-lang.org/edition-guide/introduction.html.

For this battery, performers were given the option to use either the 2021 or 2024 editions. As a result, perfomers using the 2024 edition may be required to produce code that uses more explicit unsafe blocks than a performer using the 2021 edition, however, this is generally considered desirable as the explicit use of unsafe blocks gives clarity as to precisely where in a function an unsafe operation may occur. Additionally, the 2024 edition requires the `no_mangle` attribute to be marked as `unsafe`, which will inflate syntactic unsafety metrics as this attribute is ubiquitous in the library test cases. In future batteries, we plan to enforce the use of the 2024 edition among all performers, but this is presently difficult as the `c2rust` tool currently targets a version of Rust that predates the 2024 edition. However, development to support the 2024 edition is in progress.

Undefined Behavior Undefined behavior (UB) in C[8] takes many forms, from classical memory safety errors to more nuanced issues like the order arguments are evaluated in a function call. For the purposes of this evaluation, we consider exploitable forms of undefined behavior, and in particular focus on buffer and integer overflows (or underflows). Removing vulnerabilities is a key motivation for translating to Rust, so here we assess how well performers' translators are handling commonly exploited forms of UB.

To assess how performers dealt with UB, the key question is – did translators recognize the UB, and if so how did they deal with it? We break this question down into two parts. The first is whether to preserve the application's semantics, or alter them, e.g., by skipping inputs that would result in a memory corruption, or by adding in manual error handling. The second key decision is whether to translate to safe code or unsafe code. Safe code provides runtime checking to detect UB — though some checks on arithmetic operations are only present when the code is compiled in debug and not release mode[9]. Unsafe code does not typically provide detection of UB.

When evaluating whether semantics are preserved, the worst option is to silently change the semantics of a function. While doing so would ideally be caught by correctness tests, it is nonetheless dangerous. A better option is preserving the UB, which while it decreases the safety gained from translation at least has the property of matching behavior. A competing approach is to add some type of error handling – either explicitly in the code, likely resulting in an error message and a crash, or relying on Rust's built in error handling by translating to safe code. Overall, the best way to handle UB is unclear, though changing the behavior of the application without developer intervention seems to be the most dangerous option.

Note that this assessment is informational only, and performed to give insight into how well performers handle common forms of UB, without being formally scored as part of the evaluation.

We assess performers' translation of C code containing UB on three tests cases, two synthetic and one organic. The synthetic test cases chosen were `009_stack_buffer_overflow` (Listing 1), `010_integer_overflow` (Listing 2), and the organic test case was `hdr_bitrate_lib` (Listing 3). These cover a stack-based buffer overlow, and integer overflow, and a buffer underflow respectively, all commonly exploited forms of UB.

[8] See the ISO WG14 report on UB [39].

[9] Tooling also exists to catch errors during intermediate stages of compilation, for instance Miri [40].

The stack-based buffer overflow test case, Listing 1, converts a user input to an int (line 11) and then uses that as an array index (line 23). The code checks the lower bounds of the array (`data > 0` — line 21), however the upper bound, i.e., `data < 10` is not checked, allowing a buffer overflow.

The integer overflow test case, Listing 2, reads user input into a `char` variable at line 12. On our system using `clang`, `char` is signed — note that on other architectures it may be unsigned. Because `char` is a signed type on our system, it is susceptible to overflow. At line 14, 1 is added to this value, leading to an overflow if the input value was MAX_CHAR (as defined on the test system).

The buffer underflow test case, listing 3, calculates the element of a three dimensional array to return based on the argument given to the function. The second index is calculated by subtracting 1 at line 13. If the value prior to the subtraction is 0, this leads to the index being out of bounds — MAX_UINT8 in this instance.

```
1  void bad()
2  {
3      int data;
4      /* Initialize data */
5      data = -1;
6      {
7          char inputBuffer[14] = "";
8          if (fgets(inputBuffer, 14, stdin) != NULL)
9          {
10             /* Convert to int */
11             data = atoi(inputBuffer);
12         }
13         else
14         {
15             printLine("fgets()␣failed.");
16         }
17     }
18     {
19         int i;
20         int buffer[10] = { 0 };
21         if (data >= 0)
22         {
23             buffer[data] = 1;
24             /* Print the array values */
25             for(i = 0; i < 10; i++)
26             {
27                 printIntLine(buffer[i]);
28             }
29         }
30         else
31         {
32             printLine("ERROR:␣Array␣index␣is␣negative.");
33         }
34     }
35 }
```

Listing 1: Function from test 009 with stack buffer access potentially out of bounds at line 23, the check at line 22 checks the lower bound of the array but not the upper.

```c
#include <stdio.h>

void printHexCharLine (char charHex)
{
    printf("%02x\n", charHex);
}

int main()
{
    char data;
    data = '␣';
    fscanf (stdin, "%c", &data);
    {
        char result = data + 1;
        printHexCharLine(result);
    }

    return 0;
}
```

Listing 2: Test case 010 with potential integer overflow at line 14 if the input is MAX_CHAR

```c
#include "lib.h"

unsigned hdr_bitrate(const uint8_t *h) {
    static const uint8_t halfrate[2][3][15] = {
        {{0, 4, 8, 12, 16, 20, 24, 28, 32, 40, 48, 56, 64, 72, 80},
         {0, 4, 8, 12, 16, 20, 24, 28, 32, 40, 48, 56, 64, 72, 80},
         {0, 16, 24, 28, 32, 40, 48, 56, 64, 72, 80, 88, 96, 112, 128}},
        {{0, 16, 20, 24, 28, 32, 40, 48, 56, 64, 80, 96, 112, 128, 160},
         {0, 16, 24, 28, 32, 40, 48, 56, 64, 80, 96, 112, 128, 160, 192},
         {0, 16, 32, 48, 64, 80, 96, 112, 128, 144, 160, 176, 192, 208, 224}},
    };
    return 2 *
           halfrate[!!((h[1]) & 0x8)][(((h[1]) >> 1) & 3) - 1][((h[2]) >> 4)];
}
```

Listing 3: hdr_bitrate_lib test. Note that the second index in the access at line 13 is calculated by subtracting 1, potentially leading to an out of bounds index.

4.1.3 Performance

Translation Runtime The time it takes to translate a C project to Rust is important for a translator's adoption. We capture this metric[10] in terms of hours it takes to translate 1,000 lines of C code (hours / KLOC). We capture this at the granularity of an AWS job, not the absolute time a translator took to complete. As a result, this metric includes other processing time like downloading and uploading to/from S3, and other environment setup. This might lead to fluctuations in perceived runtimes, though is standardized across all translators. This will be further improved in future evaluations.

Runtime Overhead Rust adopts the philosophy of zero-cost abstractions, or useful abstractions that negligibly impact runtime. Successful translations into Rust should exploit this principle to limit the increased time it takes to execute the same workload in C. We evaluate this using `perf-stat`[11] [41]—a common tool for gathering performance counter statistics for C programs, that can also be applied to Rust. This captures several key statistics: instructions executed, CPU cycles, absolute runtime, user runtime, task clock, cache references/misses, and many others that are used in our evaluation.

Memory Overhead Similarly (though not as important on modern hardware where memory is plentiful) is the goal of a Rust program's memory footprint to be comparable to the original C. To support this we use another common program, `valgrind`[12] [42] (using its tool `massif`—a memory profiler). This allows us to calculate the maximum number of bytes allocated to both the heap and stack, along with the bytes allocated as metadata for the memory allocator.

4.1.4 Idiomaticity

Idiomaticity in Rust refers to the extent to which code follows established Rust conventions and leverages the language's intended abstractions, such as ownership and borrowing, pattern matching, and standard library constructs. High idiomaticity is important because it improves readability, maintainability, and safety, and it allows the code to better benefit from Rust's compile-time guarantees and ecosystem tooling.

Automated Our automated idiomaticity metrics[13] use both `rustc` errors/warnings and `clippy` [43] lints. `rustc` errors prevent a Rust program from compiling, which is already computed by our functional equivalence evaluation in section 4.1.1, but the specific reasons for each error are captured here. Warnings represent potential issues, but don't rise to the level of a failing compilation. These include unused variables, dead code, and others, which are important for idiomatic Rust code.

[10] Publicly released at https://github.com/DARPA-TRACTOR-Program/PUBLIC-Test-Corpus/tree/main/tools/translation_performance.

[11] Publicly released at https://github.com/DARPA-TRACTOR-Program/PUBLIC-Test-Corpus/tree/main/deployment/scripts/github-actions/runtests/evaluation/perf_metrics.py.

[12] Publicly released at https://github.com/DARPA-TRACTOR-Program/PUBLIC-Test-Corpus/tree/main/deployment/scripts/github-actions/runtests/evaluation/valgrind_metrics.py.

[13] Publicly released at https://github.com/DARPA-TRACTOR-Program/PUBLIC-Test-Corpus/tree/main/tools/rust_eval/static/static_evaluation.py.

`clippy` is a dedicated linter for Rust that provides additional best practices, code style, and idiomatic usage recommendations beyond that of `rustc`. There are many different lints available of which we utilize all in the 5 groups listed below. Each group has examples, though this list is far from exhaustive.

- **Correctness:** serious issues regarding correctness (either wrong or useless—no false positives)
  - `approx_constant`: floating point literals close to those already defined (e.g., 3.14)
  - `infinite_iter`: iteration that is guaranteed to be infinite
- **Suspicious:** similar to correctness but possible weird construct is intentional
  - `swap_ptr_to_ref`: calls to `core::mem::swap()` with parameter derived from pointer
  - `empty_line_after_outer_attr`: empty lines between attribute (e.g., `#[allow(dead_code)]`) and its corresponding function
- **Complexity:** suggestions to simplify constructs (make them shorter and/or readable)
  - `precedence`: identifies operations where precedence is unclear (i.e., readability would be improved with parentheses)
  - `unnecessary_cast`: casts to same type, int/float literals to integer/float types
- **Perf:** hints to increase performance
  - `cmp_owned`: conversion to owned value only for comparison
  - `large_const_arrays`: large `const` arrays that should be defined `static`
- **Style:** subject style guides for idiomatic code
  - `missing_safety_doc`: missing doc comments for `pub unsafe` functions
  - `needless_return`: use of `return` at end of block

Further, we use `Lizard` [44] to analyze the Cyclomatic Complexity (CCN) of both the original C and performer generated Rust. The CCN is a metric for understanding how many paths there are through each function. As such, it represents a reasonable proxy for complexity and readability. For example, a function with no control-flow structures (e.g., `if` statements, and `for` loops) have a CCN of 1. For each additional control-flow structure added to the function the, CCN increases by 1. We use the average CCN across all functions in the project under test.

Manual To assist in the evaluation of idiomaticity, and have a deeper understanding of the methods and styles of different translators, we carried out a manual inspection of the translated Rust for both performers, and our C2Rust/LLM baselines. We selected 10 organic test cases (6 public and 4 hidden) which we believed to be representative of important C features, and interesting translation challenges.

We graded all Rust projects against the rubric in Table 10. **Readability** is important for future developers to inherit a translated codebase to maintain and add features to it. This category includes

constructs like the use of *use* statements, and refactoring to make intentions and invariants clear to someone unfamiliar with the codebase. **Syntax** measures the use of *Rusty* syntax, which could be lower-level functions like `offset()`, `add()` or `is_null()`, along with higher level constructs like `clamp()` or `impl` blocks. Similarly, **Types** measures the use of *Rusty* types. This includes techniques like translating raw pointers that represent arrays in C into Rust slices or `Vec`, or using the `Option` type to represent nullability, though this list is not exhaustive.

TABLE 10

Manual idiomaticity rubric and the meaning of scores for each attribute

Score	**1**	**2**	**3**
Readability	Worse than original C	No worse than original C	Equal to or better than original C
Syntax	No Rusty syntax	Some Rusty syntax	Excellent Rusty syntax
Types	No Rusty types	Some Rusty types	Excellent Rusty types

Figure 1 shows three different *hello world* programs. Figure 1a, shows the C version that would be given to a translator. Figure 1b shows the expected translation into Rust—what a skilled Rust developer would do. This would get 3's across the board because it's equally as readable as the original C, uses Rusty syntax (i.e., the `println` macro), and uses Rusty types (i.e., a string literal) In contrast, Figure 1c shows the output of C2Rust. This would score a 1 in readability because it turned what could've been accomplished in 3 lines into 11, doesn't have `use` statements, and uses `unsafe`, all of which obfuscate the intent of the program. Similarly, it would score a 1 in both syntax and types because it directly uses `printf` and the C types `c_char` and `c_int`, respectively.

As with any manual evaluation, the process is subject to potential human error, ambiguity, and evaluator bias. To mitigate these effects to the highest extent possible we had conversations as a group to analyze the original C code, understand what a skilled Rust developer would do to translate it, and evaluate each translated test case including all performers and both baselines. We wrote detailed comments for each translation, and would debate internally about the specific score to give to each translation based on those comments. In the future, we foresee moving to a *Likert* (1-5) scale to decrease ambiguity in scores (e.g., a translation we felt deserved a 2.5), along with the possibility of introducing a blind procedure for producing scores with no foreknowledge of the source of the translation, to further reduce any possible bias.

```c
#include <stdio.h>
int main() {
    printf("Hello World\n");
}
```

(a) Hello world in C

```rust
fn main() {
    println!("Hello World");
}
```

(b) Hello world in Rust

```rust
#![allow(dead_code, mutable_transmutes, non_camel_case_types,
non_snake_case, non_upper_case_globals, unused_assignments,
    unused_mut)]
extern "C" {
    fn printf(_: *const libc::c_char, _: ...) -> libc::c_int;
}
unsafe fn main_0() -> libc::c_int {
    printf(b"Hello World\n\0" as *const u8 as *const libc::c_char);
    return 0;
}
pub fn main() {
    unsafe { ::std::process::exit(main_0() as i32) }
}
```

(c) Hello world from C2Rust

Figure 1. Hello world examples of C-to-Rust translations

4.2 BATTERY 01 BENCHMARK

The first test battery, B01, was designed to contain almost exclusively C programs which can be represented easily within the Rust type system. In particular, it excludes programs which store memory on the heap, excludes programs which exhibit parallel execution, and aims to exclude programs which exhibit memory access patterns that are difficult to represent in Rust's affine type system (e.g., interior mutability).

The explicit set of features included in B01 are:

- Pointers to data on the stack or static
  - No `malloc` or `void*`
  - Static initialization/constants
  - Pointer arithmetic only:
    - Into arrays
    - Using `argc` to index/parse `argv`
  - Pass-by-reference for values/structs not involving pointer arithmetic
  - Scoped arrays
- Error handling

  - `errno.h`
  - Functions that return Boolean or other values for success/failure, and put the result of computation into a referenced value
    * Many `libc` functions do this, for instance
    * Possibly including different references pointing to members of the same object (difficult for Rust ownership)

- Global state
  - Compile-time constants
  - Static variables (mutating during execution)

- Simple data structs without recursion and with size known at compile time

- Code using `libc`
  - Rust translations will require Foreign Function Interfaces (FFI) calling C unless they swap in a Rust library or attempt to translate referenced functions
  - Functions from `libc` should (mostly) have Rust analogues that map naturally

- Library functions
  - C interface to test with (as a drop-in replacement)
  - This will require unsafe code to permit FFI from C

4.3 MILESTONE PROJECTS 00 AND 01 BENCHMARKS

Following the initial battery B01 tests, the performers were tasked with translating two existing project from the open-source community. The final evaluation of both translations were carried out four months after the original evaluation allowing the performers to incorporate feedback from the T&E team and results from the initial translation tests. The two projects chosen for evaluation were

1. Milestone Project 0 (P00): A repository consisting of the Perlin noise library [45]. P00 is a compact but computation-heavy codebase that exercises floating-point arithmetic, array indexing, and performance-sensitive inner loops making it representative of numerical and signal-processing workloads commonly found in legacy C code.

2. Milestone Project 1 (P01): A repository focusing on the SPHINCS+ cryptographic library [46], which is substantially larger and more complex than P00, and stresses translation tools with extensive bit-level manipulation, strict correctness requirements, and sensitivity to even minor semantic deviations. P01 additionally tests translation tools' management of build systems producing multiple libraries and supporting multiple configurations.

Together, these milestones complement the test battery by introducing more realistic and domain-specific translation challenges.

Both projects were modified versions of open-source libraries with the original Perlin Noise library authored by Sean Barrett [45] and the original SPHINCS+ implementation authored by Eyal Ronen [47] leveraging the NIST Post-Quantum Cryptography reference implementation [**?**] along with an implementation of the BLAKE hash function [48] taken from the SUPERCOP benchmark framework [**?**]. The two projects were modified to only exercise a similar subset of C features to those in B01; however P01 underwent a significant transformation to test the translation tools' handling of C project structure and configurability.

4.4 EVALUATION INFRASTRUCTURE

The test and evaluation infrastructure[14] was designed to support the execution of translators in a consistent environment which could be replicated by performers for their own independent testing. The translation environment is instantiated using AWS Cloud Development Kit (CDK) written in Python. First the Virtual Private Cloud (VPC) and input bucket are created as a single CDK stack, and then the translator environments are instantiated with reference to the created VPC. Translation occurs as AWS Batch jobs run on Fargate using containers which are built using Docker provided by each performer team. The translation environments into which jobs are deployed all reside in a VPC and are provided external connectivity to reach out to LLM API endpoints. Each performer team's containers are given read access to an S3 bucket containing tar archives of C test cases and write access to an S3 bucket to store their Rust translations. The tar archives do not contain test vectors (sample inputs and outputs) but only the C program and the materials necessary to build that C program. Each test case contains either a top-level `CMakeLists.txt` or a `CMakePresets.json` which defines the means by which the C program should be built.

Secrets needed to access LLM API endpoints for performers leveraging LLMs are provided using the AWS Secrets infrastructure leveraging the associated AWS CDK functionality.

[14] Publicly released at https://github.com/DARPA-TRACTOR-Program/PUBLIC-aws-translate.

This page intentionally left blank.

5. EVALUATION RESULTS

This section presents the evaluation results obtained using the techniques and metrics described in the previous section. Alongside quantitative measurements, we analyze the results to highlight key trends, trade-offs, and notable observations. Our focus here is on comparing performers across different dimensions.

5.1 FUNCTIONALITY

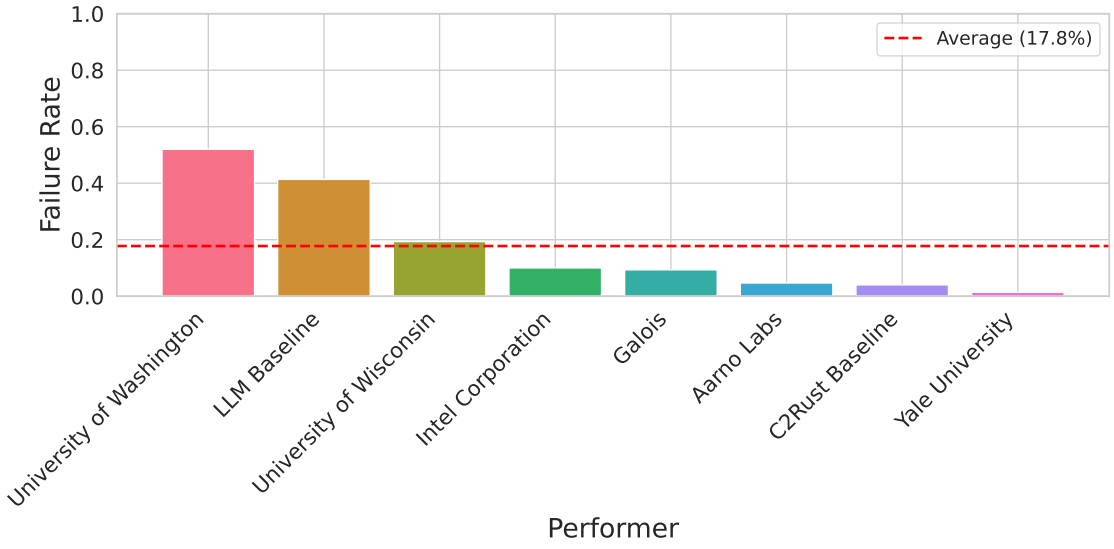


Figure 2. Overall Comparison for Correctness (Lower is Better)

Figure 2 shows a comparison of all six performer teams as well as the two baselines. The baselines are naïve use of C2Rust and naïve use of OpenAI's GPT-5.0 model with a simple prompt for translation and a secondary prompt to attempt to correct errors. This figure only accounts for overall correctness, and does not take into account any other factors such as the idiomaticity of the generated code or the unsafety of the generated code. It is interesting to see that the naïve C2Rust baseline performs competitively with the performers for this battery.

There were 150 tests in the first battery which were translated and evaluated. Figure 3 has four separate subfigures containing information across all performers. The figure shows the failure rate by performer and compares the raw proportion of incorrect outputs across each performer team. The average is plotted across the bar chart. A failed test might be the result of a failure to produce a translation, a failure to produce an

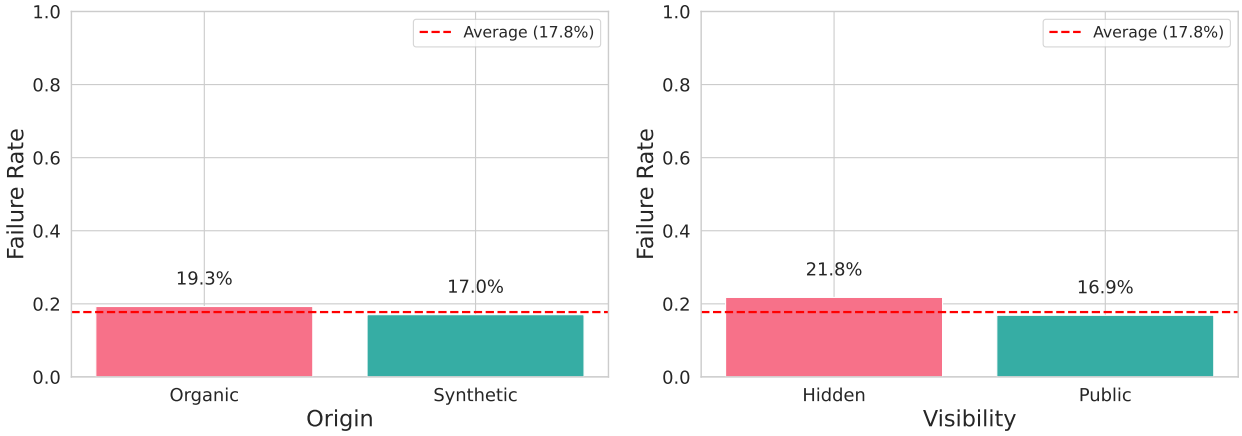


Figure 3. Failure Rate by Category Across Performers

artifact which builds successfully with ‘cargo‘, a crash at runtime, or a mismatch between the C output on a test vector and the corresponding Rust artifact.

The failure rate by origin and visibility show that performers were more likely to fail organic and hidden tests than synthetic and public tests. This intuitively makes sense as the organic tests are larger and often more complex than the synthetic tests which are intended to test translation of a specific C construct. Furthermore, the hidden tests were not available to the performers, so they had no opportunity to verify their translators work against those tests in advance of the evaluation.

The failure rate by group number figure shows the average proportion of failures in this battery overall. This figure is not particularly interesting at this stage of the program, but future reports will include comparisons of the translators’ performance against both the new tests and the existing tests separated by group or battery number. This will serve to demonstrate improvements in performance over the course of the TRACTOR program.

Figure 4 and Figure 5 shows the distribution of failures by stage for each performer. Note that this comparison does not account for the raw number of failures, it only shows the proportions for a particular performer. Again, the stages are translation, build, run, and compare.

Figure 6 shows the set of tests that performers failed with a count of the number of performers out of six which failed that test. From the figure we can see that no tests in the first battery were failed by every performer. We can also identify the test cases which were most frequently failed by performer teams.

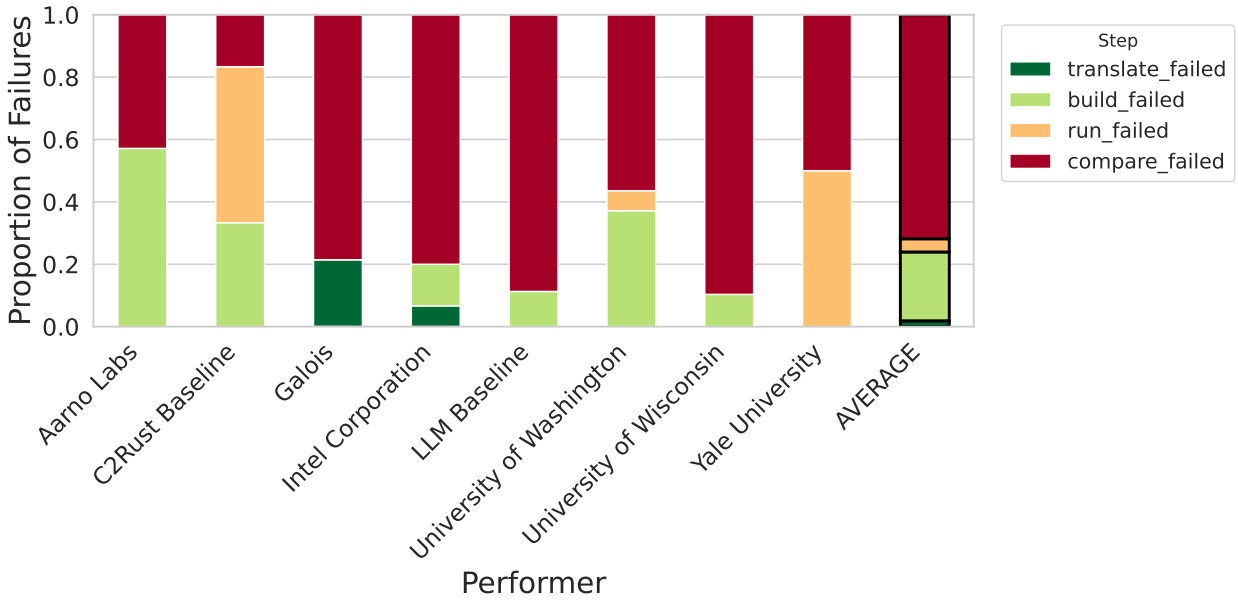


Figure 4. Proportional Stage of Failure by Performer

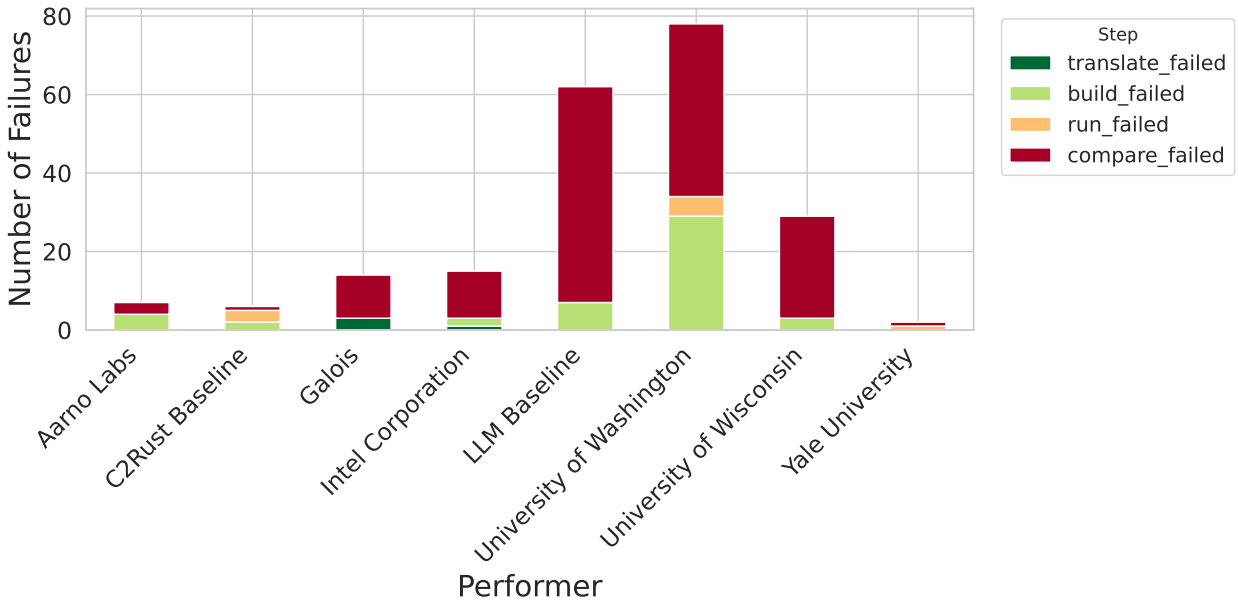


Figure 5. Absolute Stage of Failure by Performer

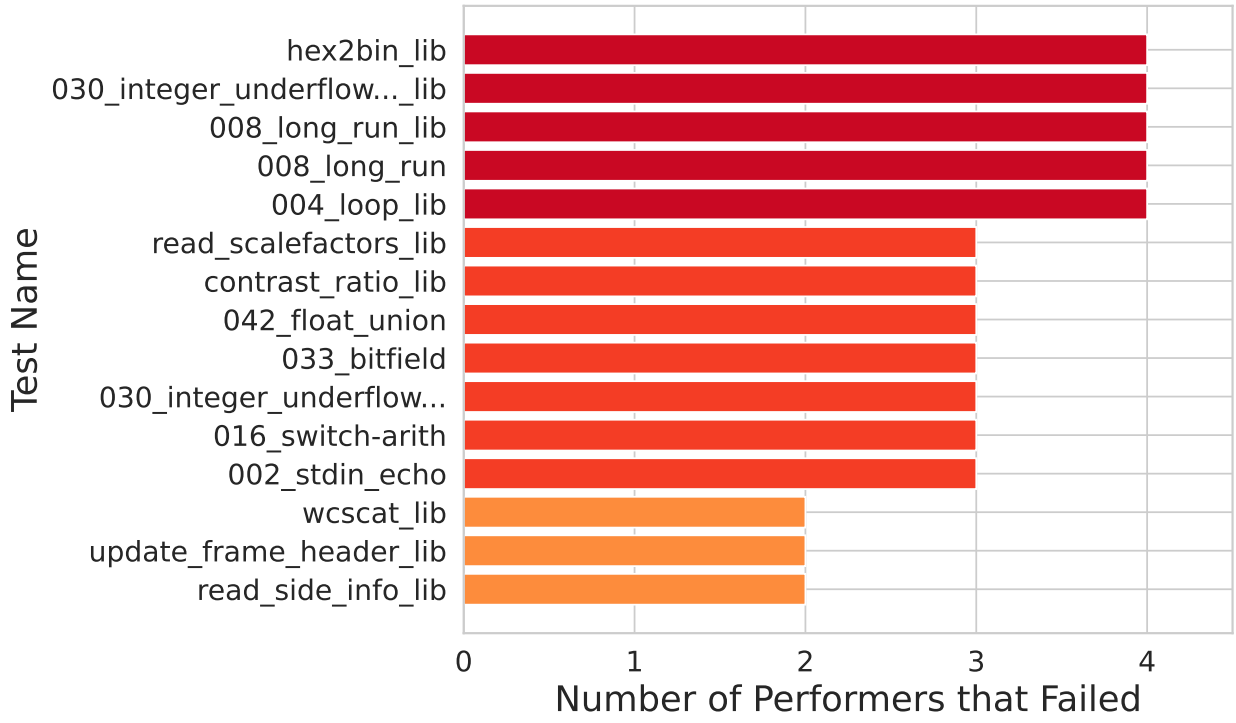


Figure 6. Test Failure Counts Across Performers (top 15 most failed test cases)

5.2 SAFETY

As one of the core unsafety metrics, and one of the more informative ones, the T&E team gathered data on which unsafe operations the performers leveraged in translation and at what *frequency*. That is, the total number of operations which were unsafe for each test case.

The performers leveraged four categories of unsafety which primarily surrounded interaction with C-style strings, conversion of C-style arrays to Rust slices[15], and interaction with raw pointers; however, some performers additionally leveraged unsafety through interaction with mutable static variables and C-style unions. A breakdown of the prevalence of each category of unsafety can be found in Figure 7 and Figure 8 with further details surrounding which unsafe functions where most used in Figure 10.

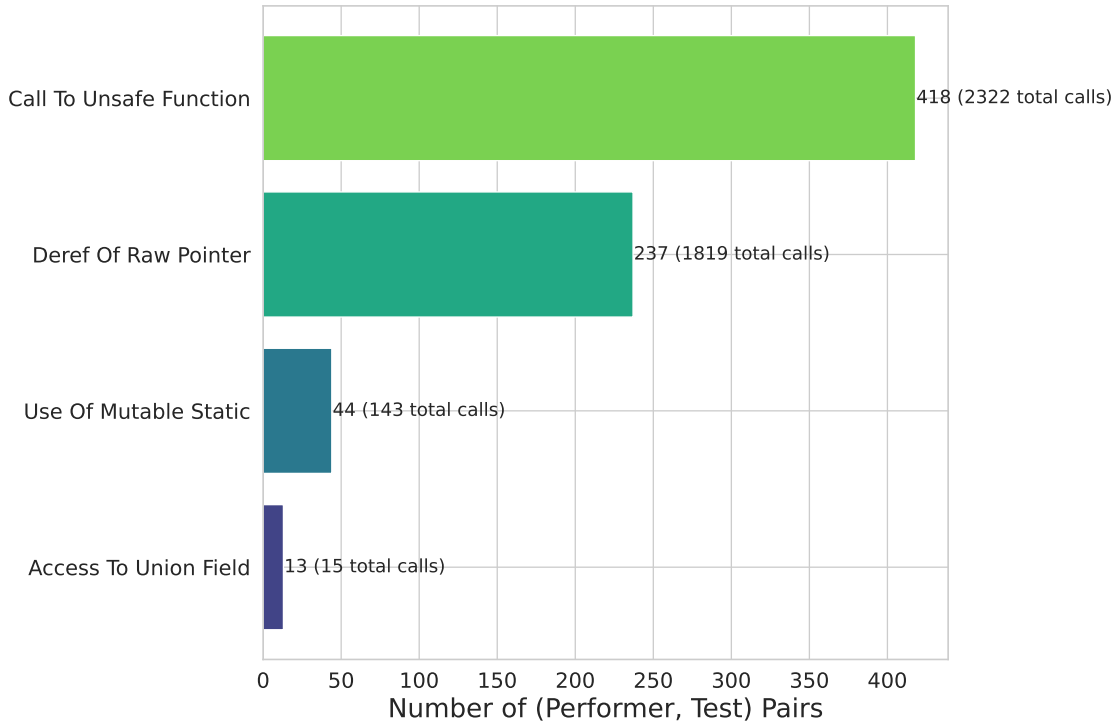


Figure 7. Kinds of Unsafe Operations Across Performers (performer-test case pairs and total calls)

[15] In Rust, slices (`&[T]` and `&mut[T]`) carry explicit length information alongside a reference to contiguous memory, enabling bounds-checked access and preventing out-of-bounds reads and writes at runtime. This contrasts with C arrays and pointers, which do not intrinsically track length. A detailed treatment of Rust's slice semantics is beyond the scope of this document; see *The Rust Programming Language* [31], Chapter 4, for an accessible introduction.

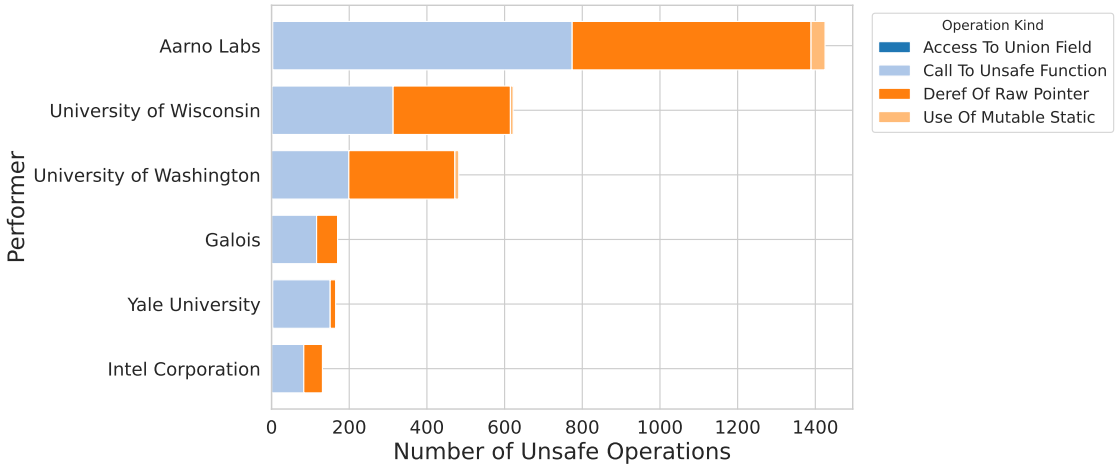


Figure 8. Kinds of Unsafe Operations by Performer (total calls of the 110 common test cases across performers)

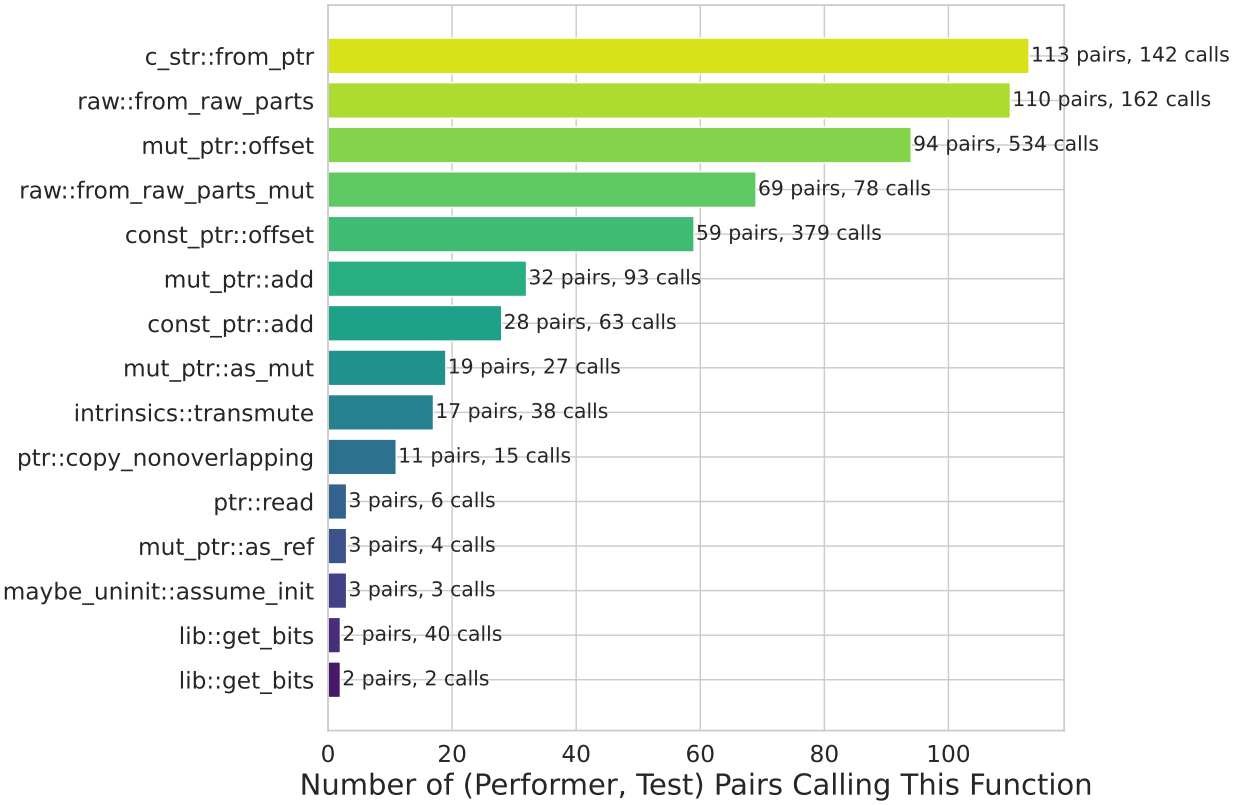


Figure 9. Unsafe Function Breakdown by Name Across Performers (performer-test case pairs and total calls)

One important point to note is that the presented unsafety data is not normalized relative to the size of the test case. Performers who sought more verbose approaches with the same unsafety *density* will necessarily have higher counts. The absolute count of unsafe operations may act as a more fair comparison at this level of granularity as the cognitive complexity of managing unsafety is largely proportional to the product of the spread of unsafety and the count of unsafety rather than its density. A comparison between and within performers can be found in Figure 10 showing how each performer team generally approached unsafety in their translations and in Figure 11 showing how each performer compares to the mean number of unsafe operations per test.

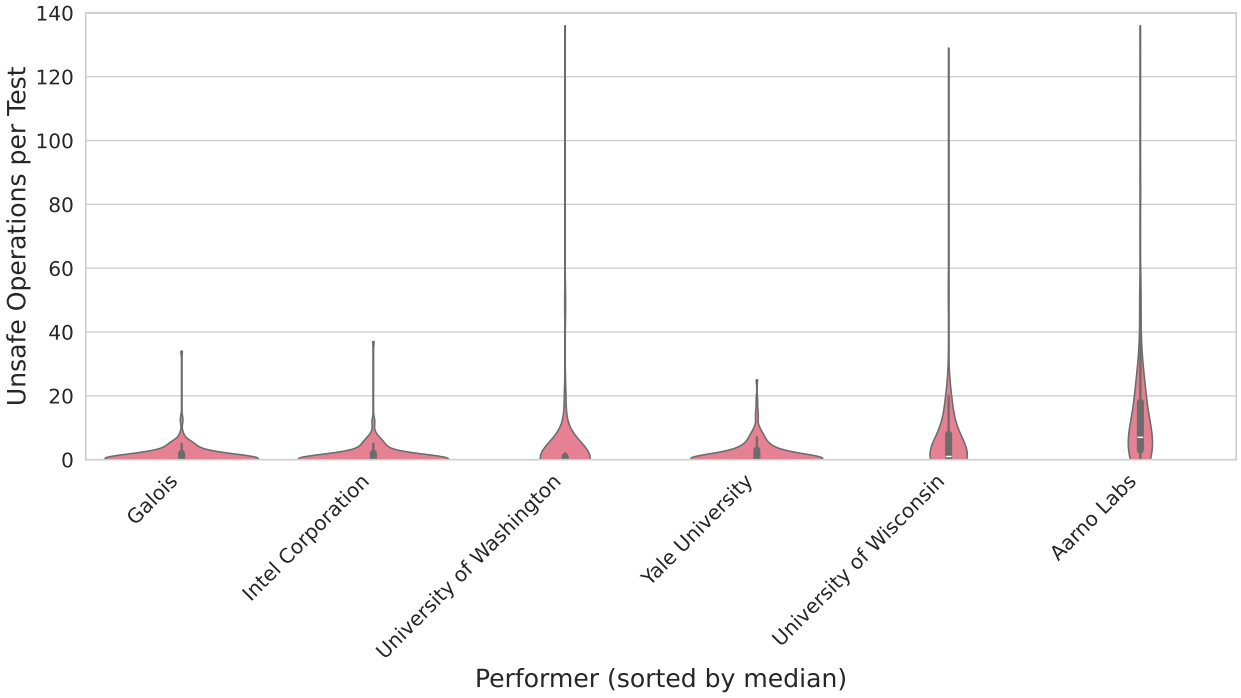


Figure 10. Unsafe Operation Count per Test by Performer (variability across tests for each performer)

It may be interesting to note that while there is a correlation between the average number of unsafe operations per test and the percent of tests that the performers were able to translate with no unsafe code other than their required C FFI, this correlation is not tight. However, this may be currently offset relative to the correctness of the resultant translations. The joint data can be found in Figure 12.

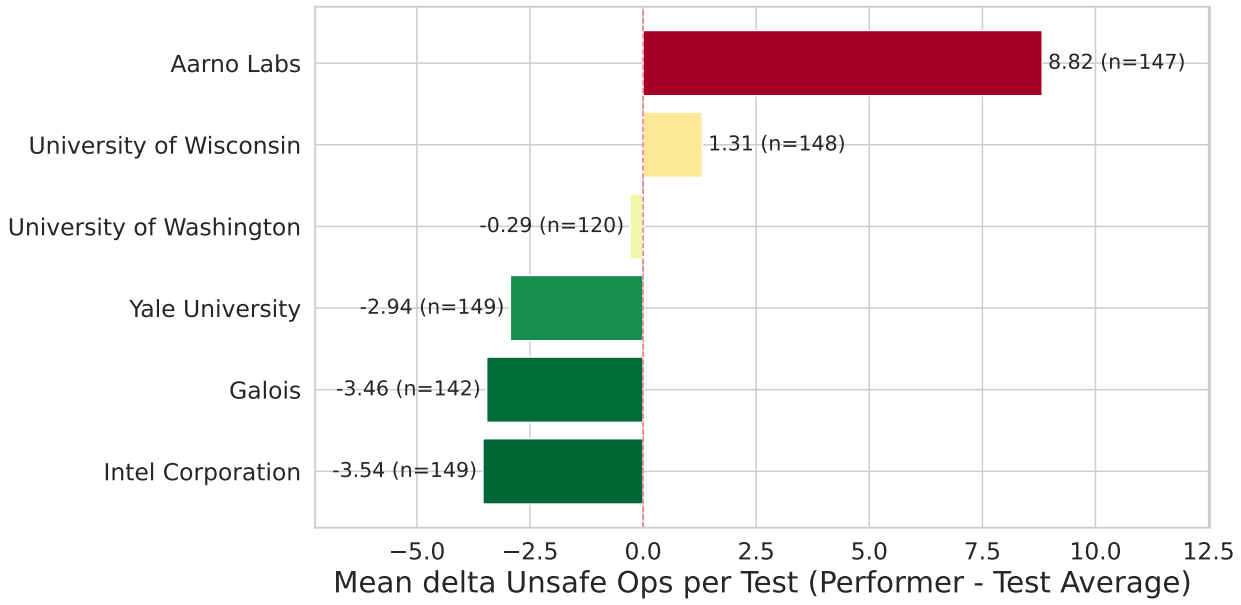


Figure 11. Comparison of Unsafe Operations vs Mean by Performer (negative means fewer unsafe operations compared to average)

Performer	Tests	Avg Ops/Test	Median	Max	% Zero Ops
Intel Corporation	150	1.55	0.0	37	59.3%
Galois	151	1.59	0.0	34	52.3%
Yale University	151	2.15	0.0	25	50.3%
University of Washington	148	3.29	0.0	136	66.9%
University of Wisconsin	151	6.35	1.0	129	40.4%
Aarno Labs	151	13.62	7.0	136	15.2%

Figure 12. Performer Summary for Unsafe Operation Use (lower average operations per test may indicate safer code generation)

Undefined Behavior For informational purposes only, we include as assessment of how performer's translation tools dealt with UB, as one of the key advantages of Rust is its safety properties, and removing vulnerabilities in the underlying C source code. Performers strategies for dealing with UB varied from entirely preserving it in unsafe blocks, to making semantic changes to patch the code, and also included raising runtime errors, either through Rust's built in safety checks or through explicitly added checks. In some instances, it appears as if the translation tools did not detect the underlying UB.

It is difficult to determine what the correct strategy for dealing with UB is. There are competing imperatives to both maintain the accuracy of the translation and correctness of the application, as well as to remove the UB. It is also unclear what the best approach is to remove UB, with options ranging from attempting to patch it to turning it into runtime errors. Regardless of whether translators chose to preserve the UB or attempt to mitigate it, tools that surface the issue through comments, runtime errors, etc. at least demonstrate that the UB was detected by the translator. This is important no matter what strategy is chosen for mitigating the UB – including the strategy of accepting it in unsafe code blocks to maintain greater equivalence with the C code.

5.3 PERFORMANCE

5.3.1 Translation Runtime

Figure 13 shows the distribution time it took to run each translation. This data is based on the wall clock duration of the AWS Batch jobs and thus includes some noise based on the time taken to copy to and from S3 buckets. Figure 14 shows the outliers for translation time. It is interesting to observe that Galois and Intel are the only teams present on the outliers listing. This may be related to these performers heavy reliance on LLMs.

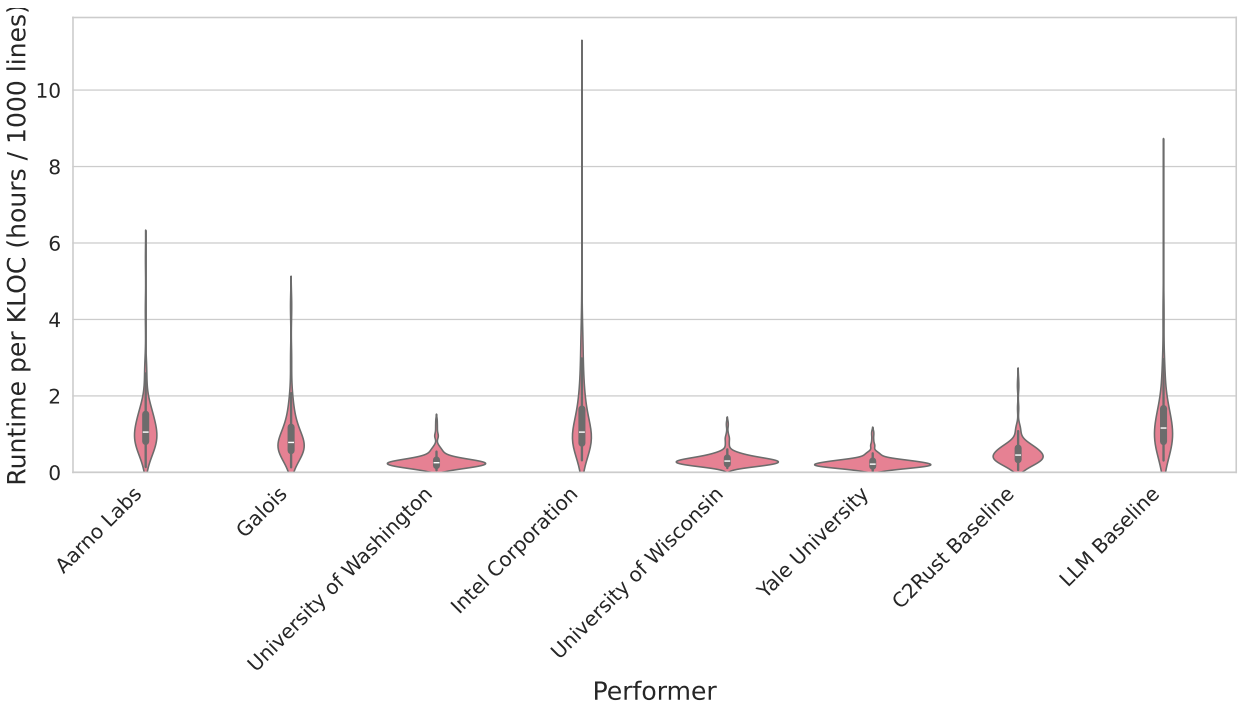


Figure 13. Translation Runtime Distribution by Performer

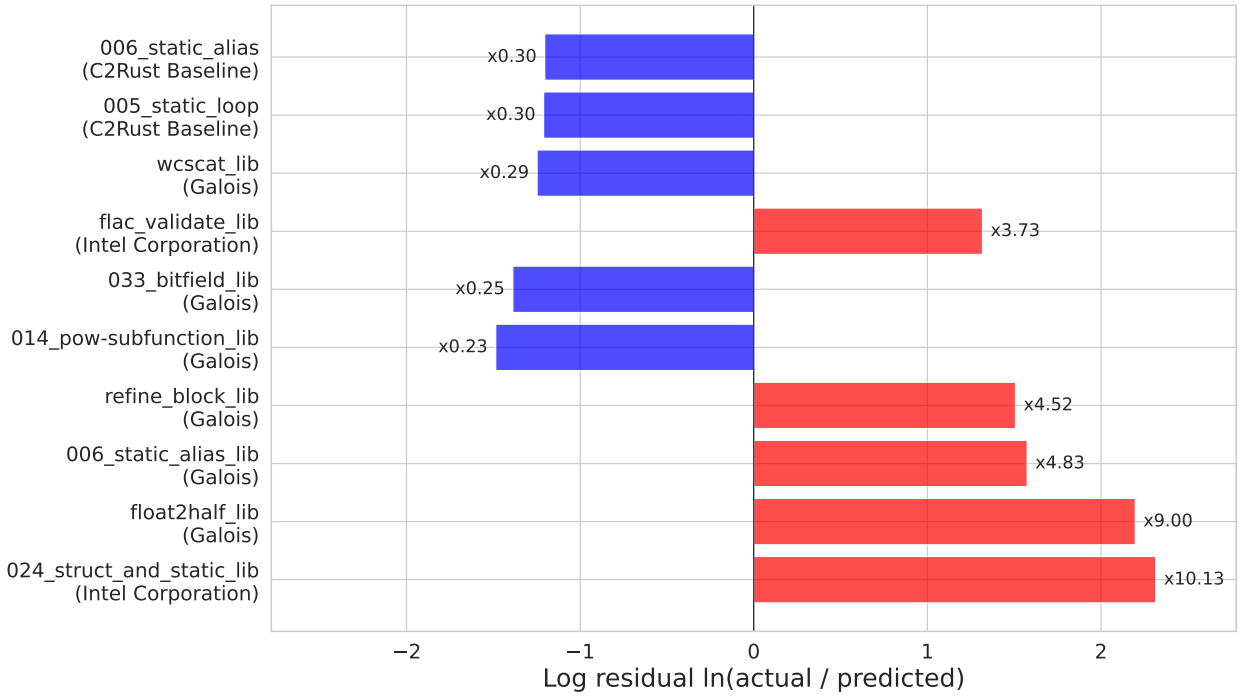


Figure 14. Outliers of Time to Run Translation as Multiplicative Error (red means slower than expected, blue means faster)

5.3.2 Runtime Overhead

Figure 15 shows the runtime execution overhead of each performer compared to the baseline C. Table 11 shows this same information in further detail. Two interesting takeaways can be observed: 1) performers runtime overheads are similar to each other—no one stood out all hovering around the average 2) modest runtime overheads are imposed by these translations into Rust compared to the original C.

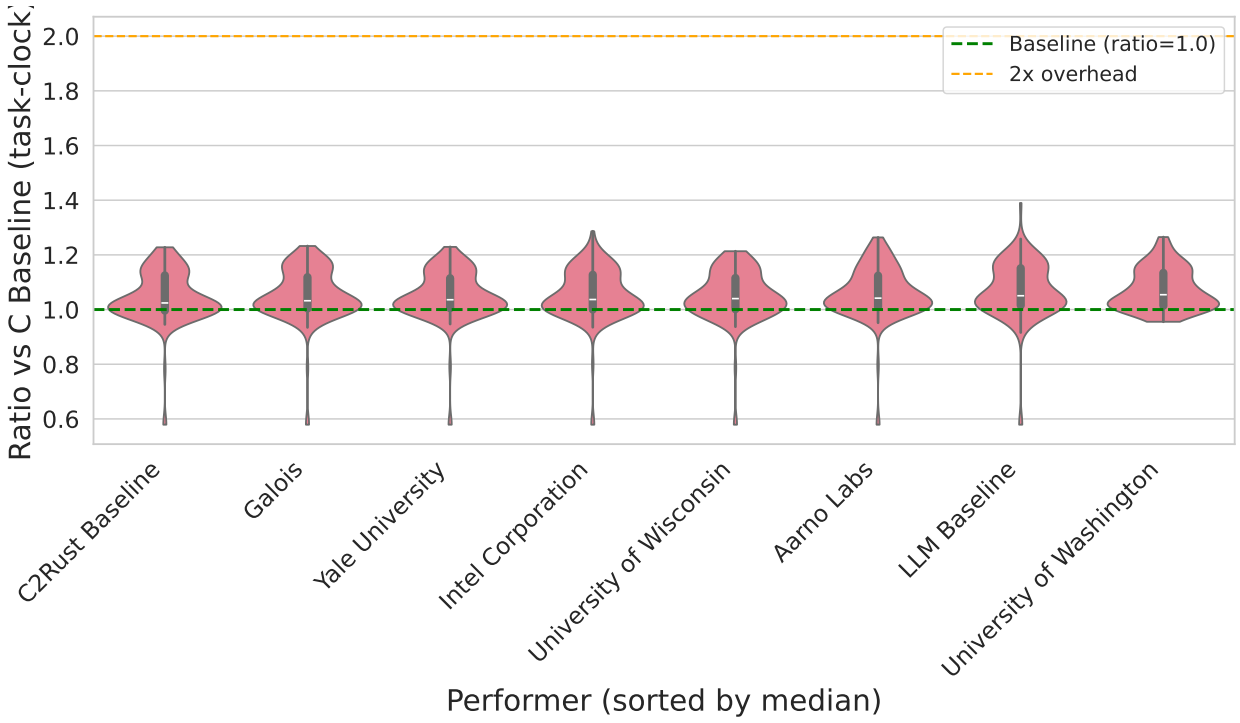


Figure 15. Runtime Overhead Distribution by Performer (ratio greater than 1 means slower than C Baseline)

TABLE 11

Runtime overhead summary compared to baseline C by performer

Performer	N	Median	Mean	GeoMean	Min	Max	<1x	Mean Ratio Delta
Aarno Labs	144	1.04x	1.06x	1.06x	0.59x	1.26x	17.4%	0.004x
Galois	139	1.03x	1.05x	1.05x	0.59x	1.23x	20.1%	-0.002x
Intel Corporation	142	1.04x	1.05x	1.05x	0.59x	1.29x	25.4%	-0.002x
University of Washington	91	1.05x	1.07x	1.07x	0.96x	1.27x	12.1%	0.017x
University of Wisconsin	140	1.04x	1.05x	1.04x	0.58x	1.21x	25.0%	-0.007x
Yale University	146	1.04x	1.05x	1.05x	0.58x	1.23x	20.5%	-0.006x
LLM Baseline	123	1.05x	1.07x	1.06x	0.59x	1.39x	14.6%	0.014x
C2Rust Baseline	143	1.02x	1.05x	1.04x	0.58x	1.23x	28.7%	-0.011x

5.3.3 Memory Overhead

Similarly, Figure 16 and Table 12 show the memory overhead imposed for each performer compared to the C baseline. We again observe a trend of modest memory overhead in the translated Rust, and little separation on a per-performer basis.

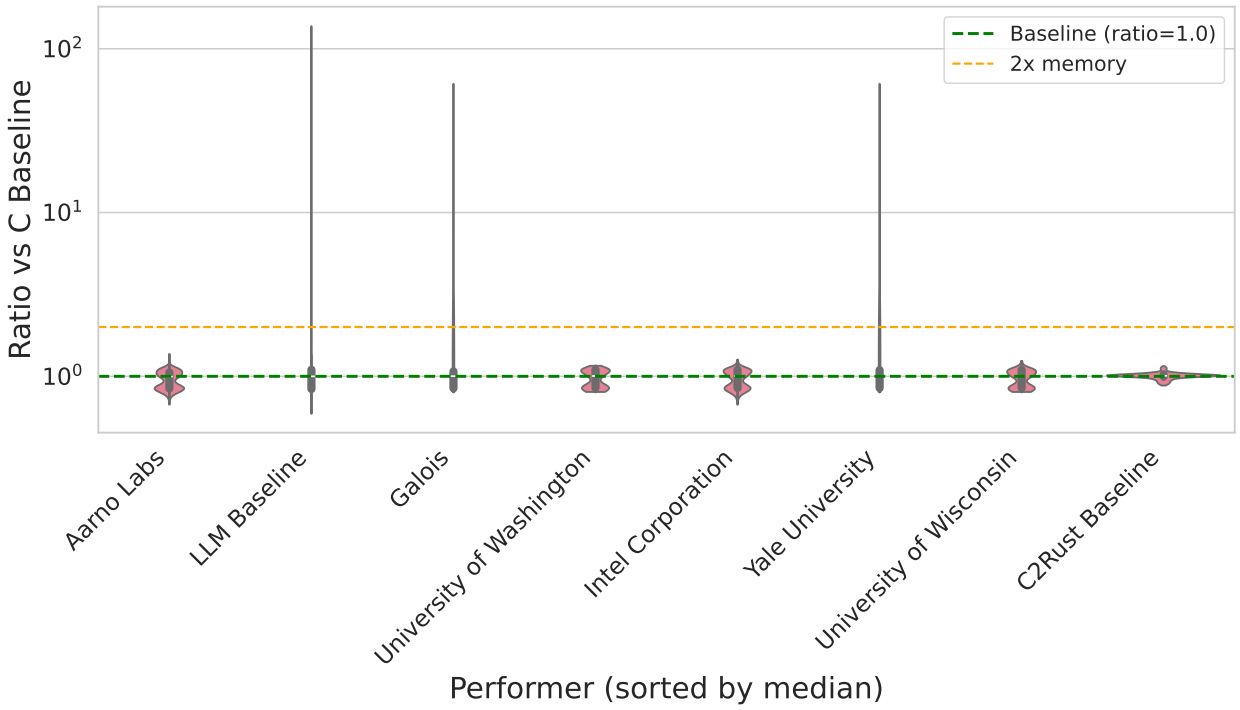


Figure 16. Memory Overhead Distribution by Performer

TABLE 12

Memory overhead summary compared to baseline C by performer: Ratio greater than 1 means more memory than C baseline

Performer	N	Median	Mean	GeoMean	Min	Max	<1x	Mean Ratio Delta
Aarno Labs	144	1.00x	0.96x	0.95x	0.68x	1.34x	52.8%	-0.25x
Galois	139	1.00x	1.39x	0.99x	0.80x	61.12x	49.6%	0.18x
Intel Corporation	142	1.01x	0.97x	0.97x	0.68x	1.26x	47.9%	-0.24x
University of Washington	91	1.00x	0.97x	0.97x	0.80x	1.16x	47.3%	-0.24x
University of Wisconsin	140	1.01x	0.97x	0.97x	0.80x	1.24x	47.1%	-0.24x
Yale University	146	1.01x	1.38x	1.00x	0.80x	61.23x	47.3%	0.17x
LLM Baseline	123	1.00x	2.08x	0.99x	0.59x	137.64x	50.4%	0.86x
C2Rust Baseline	143	1.01x	1.01x	1.01x	0.88x	1.15x	30.1%	-0.21x

5.4 IDIOMATICITY

Table 13 below compares the performers in terms of the average length and cognitive complexity of the translations the performers produce relative to each other and to the original C program. The hand-graded idiomaticity score averages across ten test cases are included in Table 14.

TABLE 13

Average Lines of Code and Cyclomatic Complexity (CCN) by Performer (lower is better)

Source	Average Lines	Average CCN
C	12.0	2.5
Aarno Labs	16.2	2.1
Galois	11.2	2.2
Intel Corporation	14.5	2.9
University of Washington	11.7	2.3
University of Wisconsin	14.9	3.0
Yale University	13.4	2.9
Avg Perf	13.6	2.5

TABLE 14

Hand-graded Idiomaticity Scores for Selected Test Cases (higher is better; score out of 3)

	Readability	Syntax	Types
Aarno Labs	1.9	1.5	1.1
Galois	2.6	2.8	2.5
Intel Corporation	2.7	2.9	2.7
University of Washington	2.8	2.7	2.4
University of Wisconsin	2.8	2.3	1.9
Yale University	2.1	2.3	2.2
Avg Across Performers	2.4	2.4	2.1
C2Rust Baseline	1.5	1.3	1.1
LLM Baseline	2.2	2.0	1.7

5.5 MILESTONE PROJECTS EVALUATION

As the two major projects for this evaluation, Project P00 "Perlin Noise" and Project P01 "SPHINCS+" were more complex than the test cases in Battery B01. The initial December evaluation of P00/P01 showed little differentiation between the individual performers' translations. As such, the T&E team provided initial feedback regarding the translation results and scheduled a reevaluation for April 2026 where the performers' updated translation tooling would be evaluated against both P00 and P01 again.

P00: Perlin Noise is an executable test case implementing a handful of operations related to Perlin noise [45]. The project under test focuses on floating point manipulation and expects performers' translations to produce output in a strict decimal format containing the same number of figures as the original C program.

P01: SPHINCS+ is, dually, an executable and a library test of a plausibly quantum resilient hash-based signature scheme [46] selected by NIST as part of the post-quantum signatures standardization process. P01 is a more complex codebase than P00 and required performers to determine which artifacts to produce and in which configurations based on a CMake context. The performers then needed to translate both the supporting libraries and a top-level driver program exercising a deterministic variant of SPHINCS+.

During the original December evaluation, no performers were successful in translating P01 for any configuration or test vector.

5.5.1 April P00/P01 Reevaluation

As there was not enough data to sufficiently differentiate performers' outputs for the two core projects evaluated in December, an additional evaluation was undertaken four months later in April 2026. The purpose of the reevalaution was to allow performers to update their translation frameworks and to allow the T&E

team the opportunity to gather data on differentiating factors in each performers' tooling. During the April 2026 reevaluation, *all performer teams* successfully passed all vectors for Project P00; the key differentiators for P00 translations were all present in the P01 translations. Because of this significant overlap, the rest of this section will be dedicated to evaluating the performers' translations of P01.

As the original Battery B01 tests did not exercise the translation tools' ability to handle multiple CMake project structures or configurations, the December evaluation did not include metrics for the terminal Cargo project structure nor the translation's handling of compile-time configurability. For the April reevaluation, the T&E team captured these additional metrics in a *manual evaluation* of the translation tools'. The new metrics focused on the translation tool's capacity to produce Cargo projects accepting configuration parameters consistent with the C source. However, for those performers who focused on single-configuration translations, the T&E team offered each performer team a choice to either translate

- A single P01 instance whose translation would need to support all possible configurations under test
- Ten P01 instances with each instance supporting a different *fixed* configuration along with the previous instance which would be used to evaluate translation configurability

Performers University of Washington and University of Wisconsin both opted to have their evaluations run against a single P01 instance, while the other four teams opted to have their evaluations run against the ten P01 instances.

Finally, to help differentiate performers along each manually evaluated axis, the April reevaluation moved from a three-point scale to a standard five-point likert scale.

5.5.2 Functionality

The correctness results for the performer translation tools are shown in Table 16. The second column from the left presents the P00 results, while the middle columns correspond to different P01 configurations. The numeric identifiers (e.g., 007 and 012) denote specific SPHINCS+ configurations.

All of the performer teams successfully translated P00 (30 of 30). For P01, most teams were able to produce working implementations for every configuration we tested. Galois and University of Washington both produced some incorrect translations. We only ran the University of Washington translator once per their request. They produced a Blake library, which crashes at runtime. Their translator also does not expose the shared objects expected to support library tests, as it combines all of the libraries produced by the C test case into a single shared object.

TABLE 15

Correctness Evaluation Scores for P00 and P01 Configurations (e.g. 007)

Performer	P00	P01 Configurations										TOTAL SUM
		007	012	014	017	028	030	036	039	045	127	
Aarno Labs	30	1	1	1	1	1	1	1	1	1	1	40/40
Galois	30	1	0	0	0	0	1	1	1	1	1	36/40
Intel Corporation	30	1	1	1	1	1	1	1	1	1	1	40/40
University of Washington	30	0	0	0	1	1	1	1	1	1	0	36/40
University of Wisconsin	30	1	1	1	1	1	1	1	1	1	1	40/40
Yale University	30	1	1	1	1	1	1	1	1	1	1	40/40

5.5.3 Safety

Figures 17, 18, 19, and 20 illustrate the types of unsafe behavior exhibited in the translated Rust. As can be seen from Table 16, there is a significant gap between performer teams in terms of the number of unsafe operations used in each test.

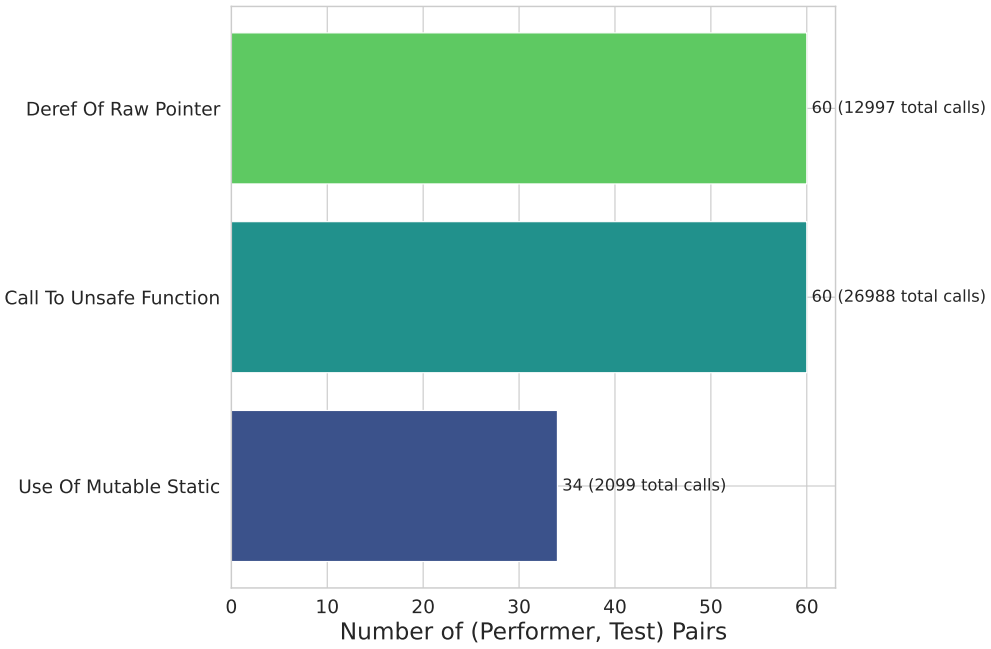


Figure 17. Types of Unsafe Operations Across Performers

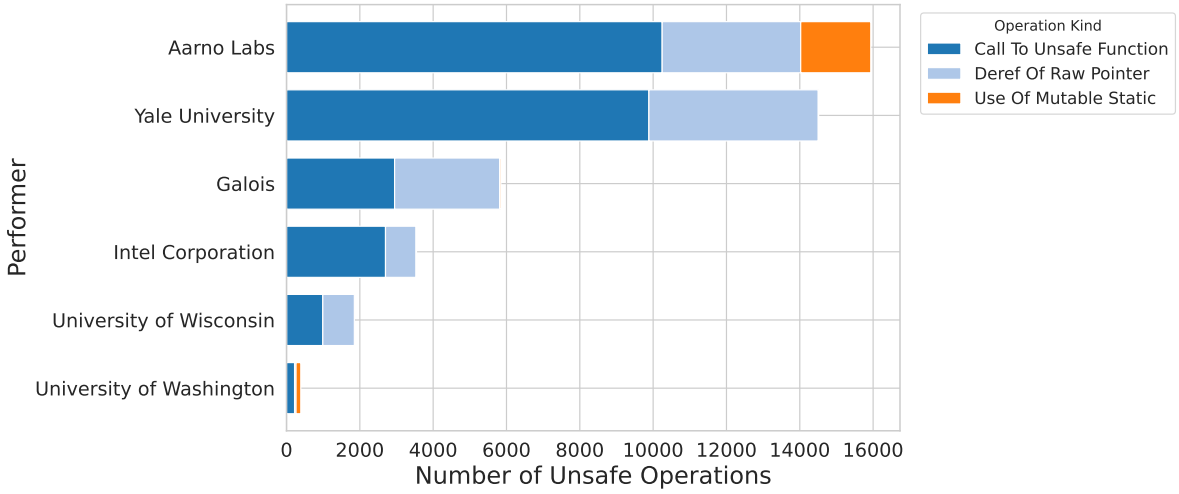


Figure 18. Types of Unsafe Operations by Performer

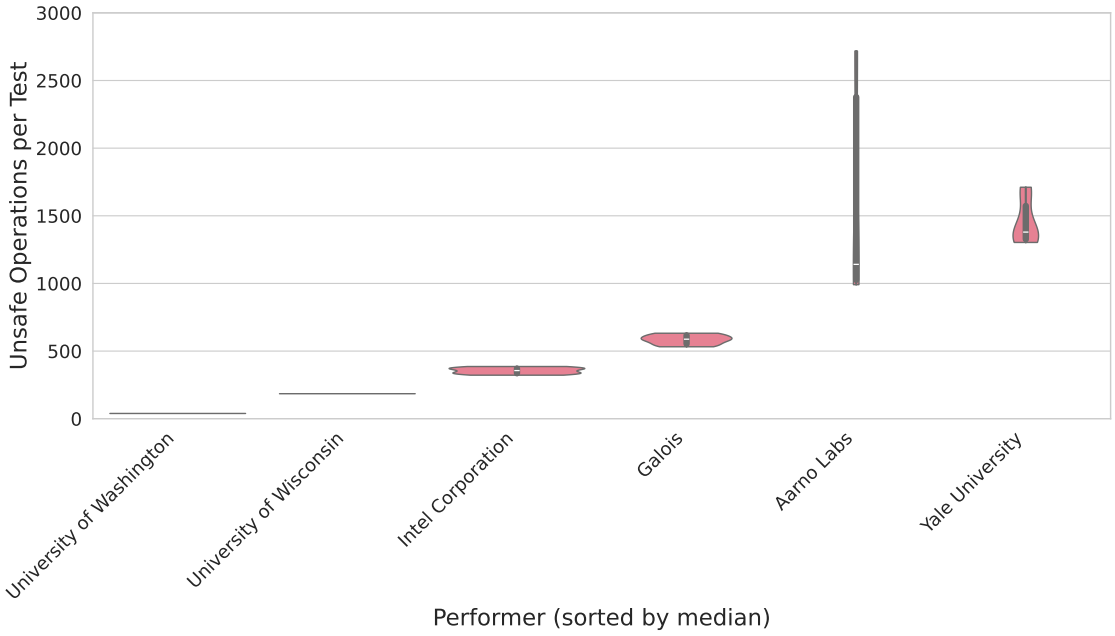


Figure 19. Unsafe Operations per Test by Performer

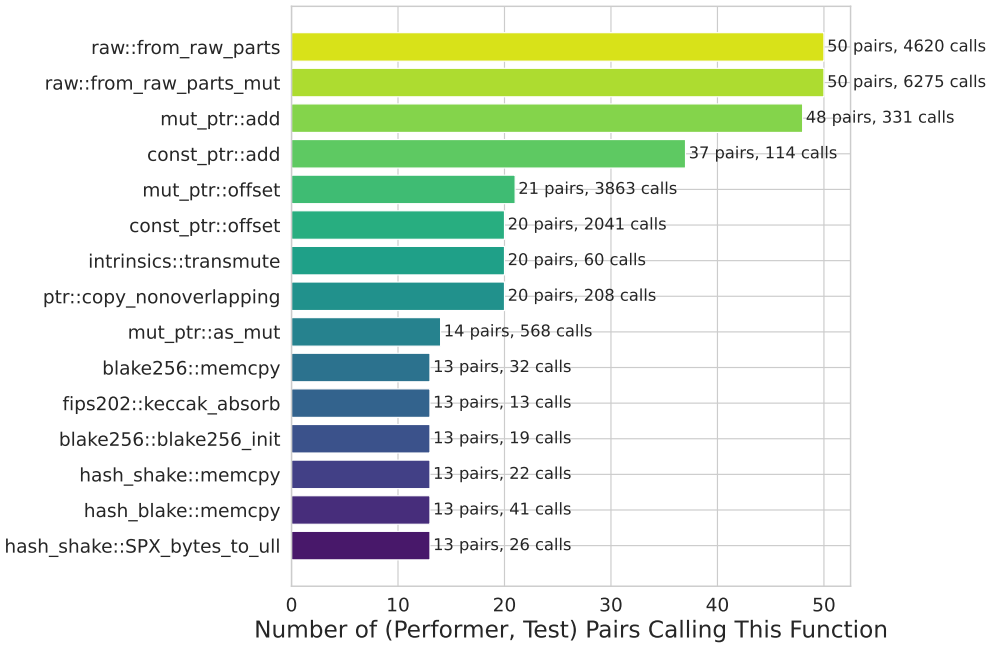


Figure 20. Common Unsafe Functions Across Performers

TABLE 16

Unsafe Operations Summary by Performer

Performer	Tests	Avg Ops/Test	Median	Max	% Zero Ops
University of Washington	10	39.00	39.0	39	0.0%
University of Wisconsin	10	185.00	185.0	185	0.0%
Intel Corporation	10	354.60	356.0	386	0.0%
Galois	10	584.80	588.0	632	0.0%
Yale University	10	1451.40	1379.5	1711	0.0%
Aarno Labs	10	1593.60	1141.0	2718	0.0%

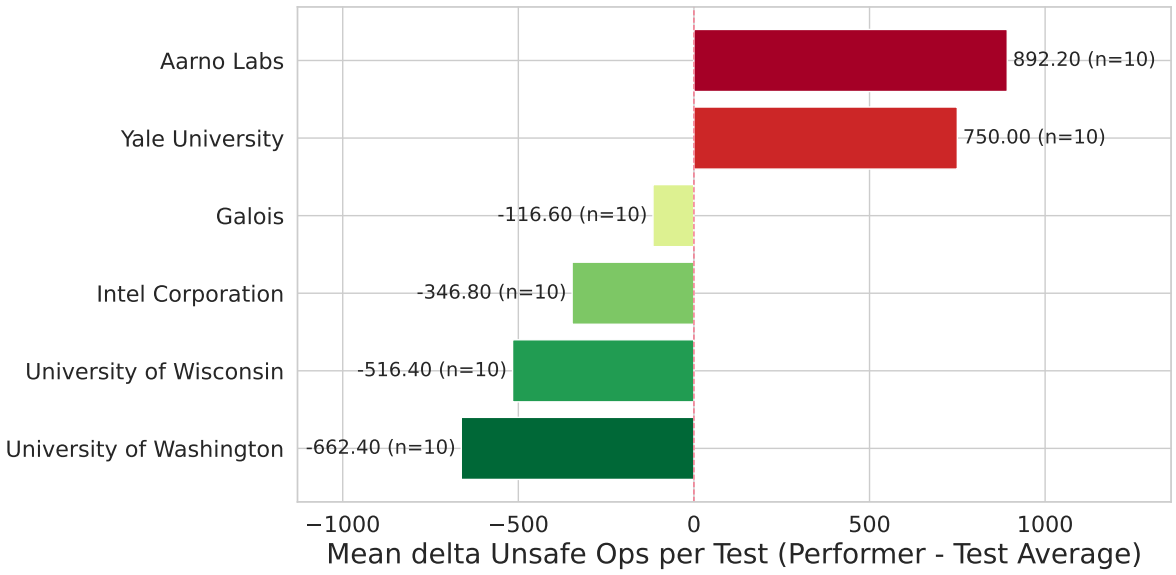


Figure 21. Unsafe Operation Deviations by Performer

5.5.4 Performance

Figures 22 illustrates the translation time for P01. Figure 23 and 24 illustrate the runtime and memory overheads of the translated code for P01.

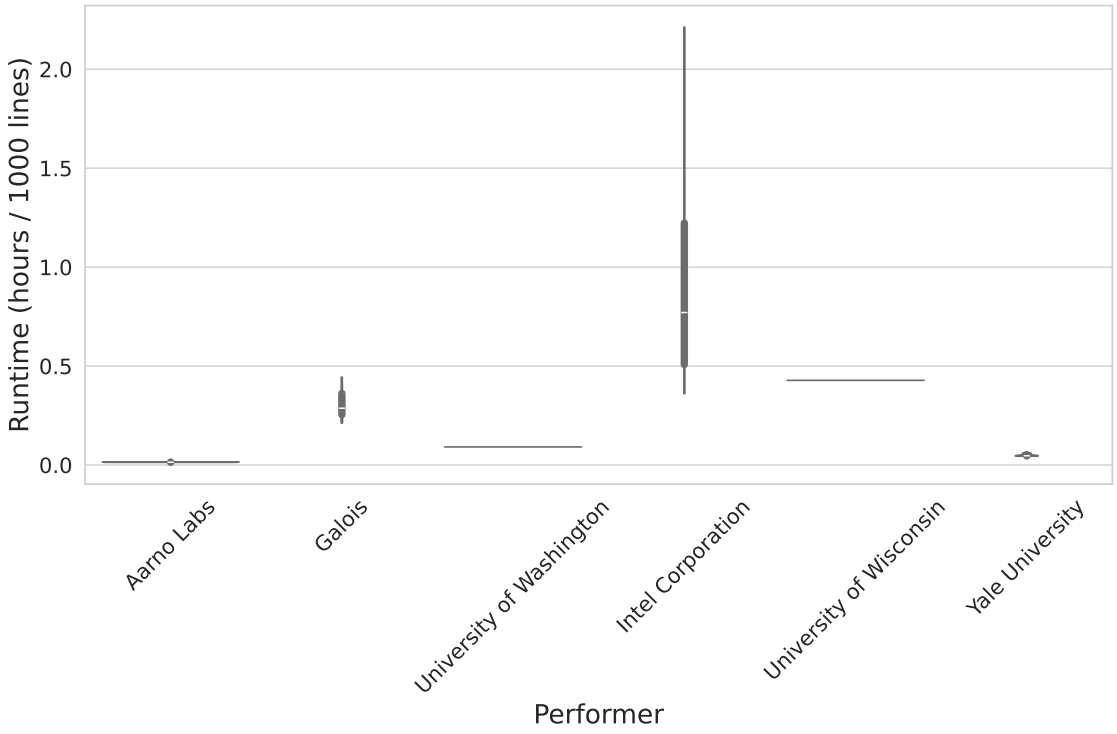


Figure 22. Translation Runtime by Performer

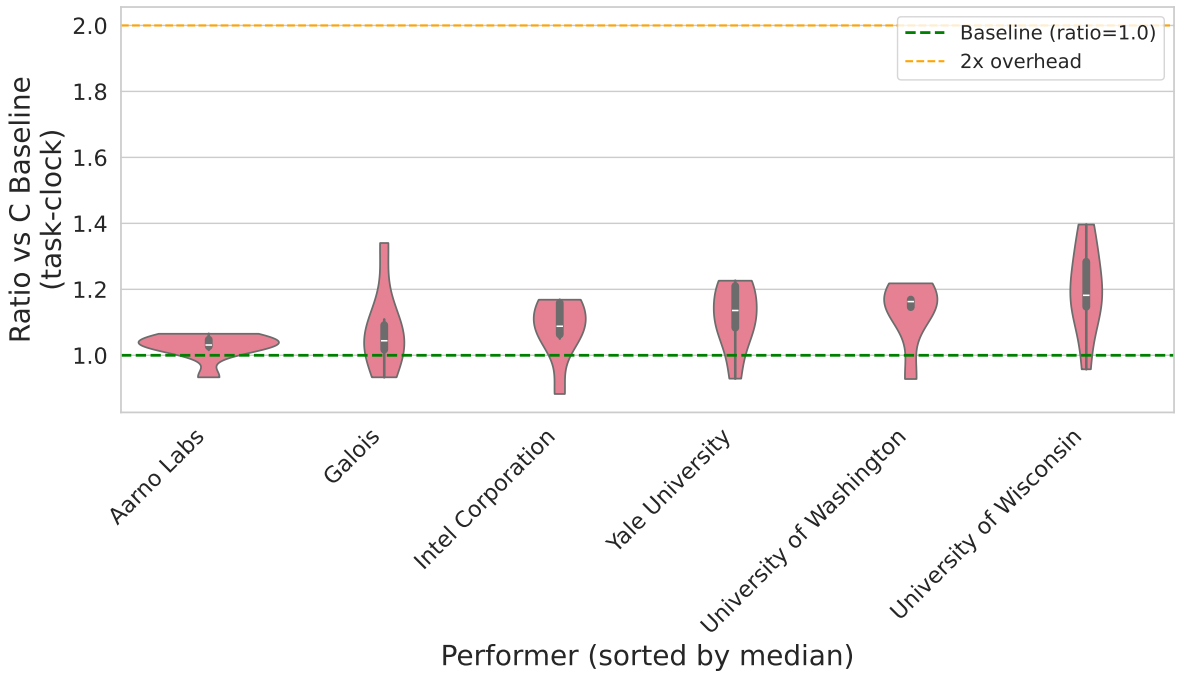


Figure 23. Runtime Overhead by Performer

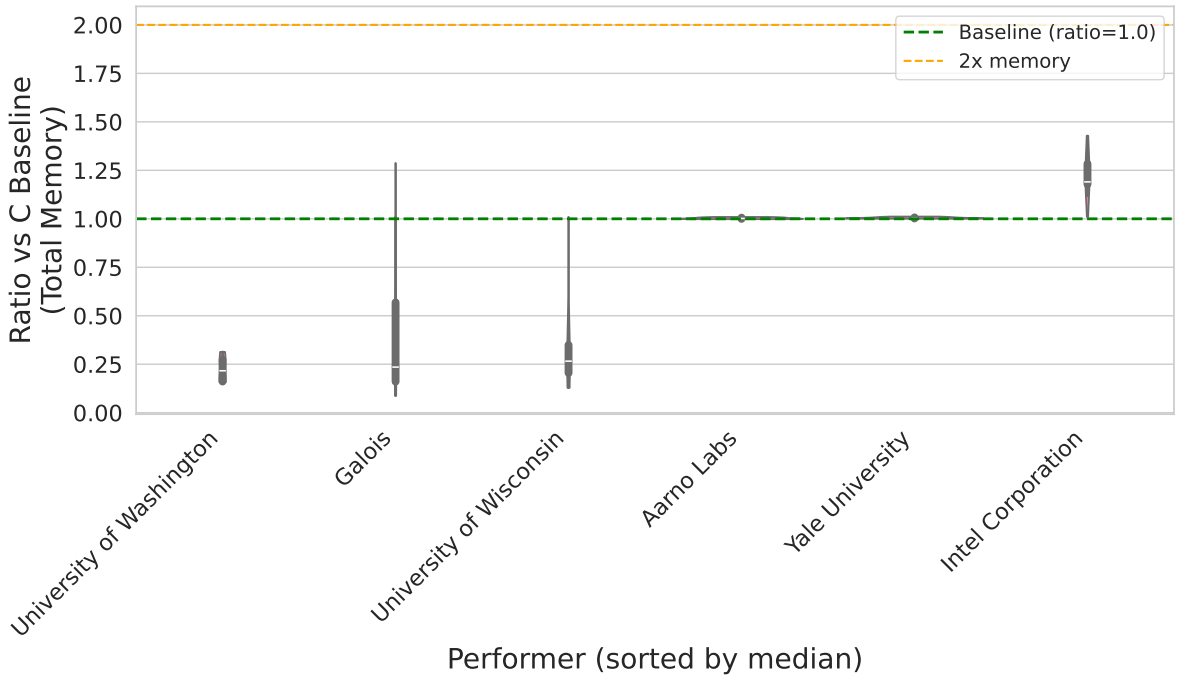


Figure 24. Memory Overhead by Performer

5.5.5 Idiomaticity

TABLE 17

Manual Idiomaticity Evaluation Score for P01

Source	Readability	Types	Syntax	Configurability	Project Structure	Average
Aarno Labs	1	1	1	1	3	1.4
Galois	2	2	3	1	2	2
Intel Corporation	4	4	4	1	4	3.4
University of Washington	5	4	5	4	3	4.2
University of Wisconsin	5	4	3	3	3	3.6
Yale University	1	1	1	2	3	1.6

TABLE 18

Average Lines of Code and Cyclomatic Complexity (CCN) by Performer (lower is better)

Source	Average Lines	Average CCN
Baseline C	3,834	2.5
Aarno Labs	4,047	2.0
Galois	4,621	1.7
Intel Corporation	9,588	3.0
University of Washington	3,983	2.1
University of Wisconsin	5,812	1.7
Yale University	4,947	2.8
Performer Avg	5,500	2.2

Table 17 shows the manual idiomaticity scores for P01. Each category is scored on a scale of 1 (worst) – 5 (best). As a general observation, teams leveraging LLM-based approaches tend to produce output that more closely resembles idiomatic Rust code. In contrast, teams whose translations rely primarily on tools such as c2rust often generate code that retains a significantly more C-like structure and style. Among the least idiomatic patterns observed were Rust functions that directly replicate unsafe C memory management operations, such as `memcpy`, rather than using Rust-native abstractions and safety mechanisms. Table 18 shows the number of lines of Rust code, and the cyclomatic complexity as an average across translations.

Figure 25 indicates that teams relying more heavily on static analysis approaches generally produced code with a larger number of Clippy lints and warnings, suggesting less idiomatic Rust output. In contrast, LLM-focused teams tended to generate code with substantially fewer Clippy issues, and in some cases none at all. Notably, both Intel Corporationand University of Wisconsinachieved zero Clippy lints or warnings. Galoiswas a significant outlier, producing 14,664 lints and warnings, the majority of which were related to operator precedence readability concerns.

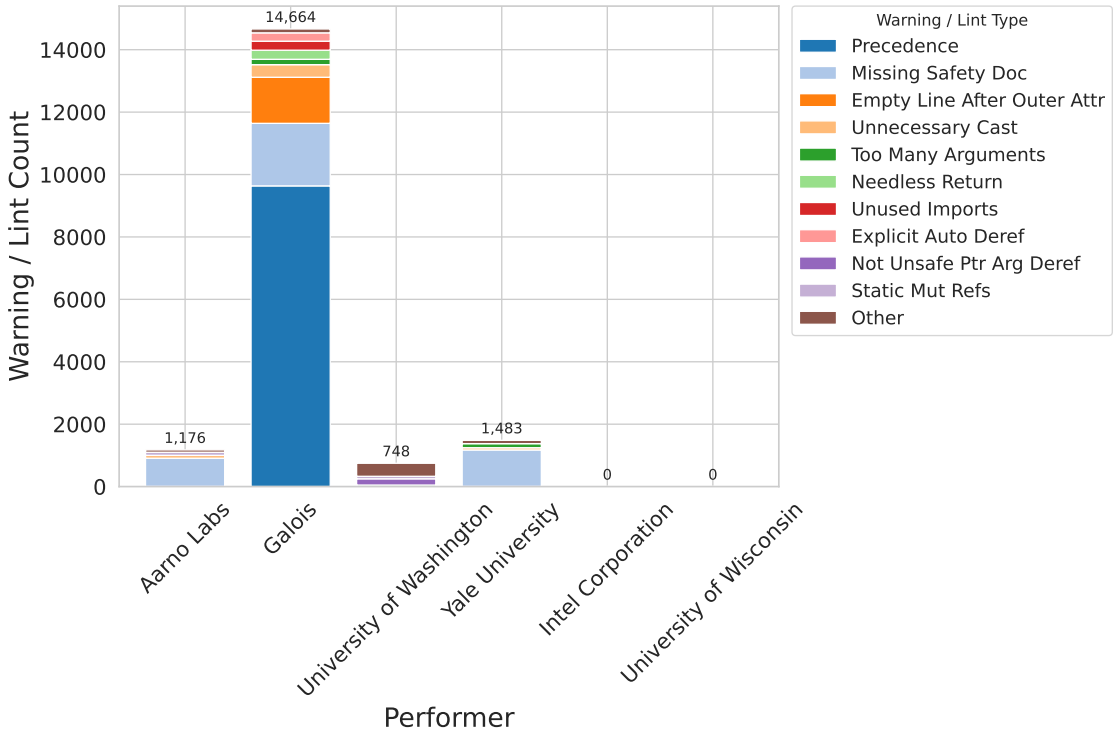


Figure 25. Lint / Warning Count by Performer

This page intentionally left blank.

6. OBSERVATIONS AND INSIGHTS

In this section, we review the main insights and observations from our evaluation. These insights were developed through a comparative analysis of six distinct translation systems evaluated under a common, controlled test and evaluation framework. By examining where and how different approaches succeeded or failed across functionality, safety, performance, and idiomaticity, the evaluation was able to surface recurring patterns, trade-offs, and complementarities that were not apparent from any single system in isolation.

6.1 PIPELINE ARCHITECTURE, ORCHESTRATION, AND CONTROL

Prompting strategies converge on a shared control structure Despite surface-level diversity in prompting styles, formats, and model choices, effective LLM-based translators converge on a common control structure: scoped context, non-negotiable behavioral constraints, explicit feedback loops, and strict, machine-ingestible output contracts. This convergence indicates that prompt engineering for translation has evolved into a disciplined design space rather than an ad hoc activity, and helps explain why observed performance differences are driven more by orchestration decisions than by prompt phrasing itself.

Prompts function as software artifacts The most robust systems treat prompts as versioned, testable, and iteratively refined interfaces, akin to APIs between orchestration logic and code generation. This framing enables systematic improvement and reduces the brittleness typically associated with LLM-based tools.

Translation granularity is a primary architectural trade-off Whole-project translation maximizes global coherence but increases context pressure and obscures the provenance of errors, while granular or dependency-ordered translation localizes failures and improves debuggability at the cost of more complex orchestration. The report shows that neither approach dominates, and that hybrid strategies can capture benefits of both.

Performance variability correlates more with orchestration than with model choice Translation runtime shows significant variance both within and across performers, with outliers often tied to orchestration decisions such as iteration depth and feedback loops rather than model capability alone. This suggests that pipeline control logic is now a first-order performance concern.

6.2 CORRECTNESS, EQUIVALENCE, AND EVALUATION LIMITS

Static Analysis and LLM Approaches Exhibit a Correctness–Idiomaticity Tradeoff. Evaluation results reveal a clear tradeoff between correctness and idiomaticity. Static analysis-based approaches more consistently generate functionally correct code, but the resulting output often retains a C-like structure and is less idiomatic in Rust. In contrast, LLM-based approaches tend to produce more natural, Rust-idiomatic code, but with lower functional reliability. These findings suggest that hybrid approaches combining static analysis with LLM-based generation may offer the most effective end-to-end translation solution.

Test-based equivalence is the practical boundary of correctness Across all systems, compilation success is necessary but insufficient to ensure semantic fidelity. Test-based equivalence emerges as the most reliable operational definition of correctness, even while the report acknowledges its limitations in the presence of incomplete test coverage. We encourage performers to develop more sophisticated notions of equivalence for self-testing of their translators.

Correctness failures are dominated by late-stage semantic mismatches Across performers, the majority of failures occur at the comparison stage rather than during translation or compilation. This indicates that modern translators are largely capable of producing buildable Rust, but subtle semantic mismatches remain the primary barrier to correctness.

Organic and hidden tests expose fundamentally different weaknesses Failure rates are consistently higher on organic and hidden tests than on synthetic or public ones, suggesting that translators overfit to visible patterns and face more difficulty with realistic composition effects. This highlights the importance of evaluation regimes that include withheld and organically derived workloads.

Baseline tools remain competitive under constrained semantics Both naïve C2Rust and naïve LLM baselines perform better than expected on the constrained first battery, particularly for functional correctness. This suggests that early-stage benchmarks may underestimate the difficulty of full-scale translation and should evolve rapidly to stress deeper semantic issues.

Evaluation metrics shape translator behavior Because correctness is primarily measured through test outcomes, translators implicitly optimize for test-passing behavior, sometimes at the expense of safety clarity or semantic transparency. This reinforces the importance of evolving metrics to better balance correctness, safety, and diagnosability.

6.3 SAFETY, UNDEFINED BEHAVIOR, AND SEMANTIC RISK

Semantic risk is modeled implicitly and differently by each team Each translator encodes a distinct notion of what is "most dangerous" to get wrong—unsafe code, silent semantic drift, ABI breakage, or overflow behavior—and optimizes accordingly. These differing risk models explain why failures are distributed unevenly across test cases rather than concentrated on a common set.

Unsafe code is often preserved intentionally rather than eliminated accidentally In multiple systems, unsafe blocks are introduced deliberately to preserve C semantics, especially in the presence of undefined behavior. This reflects a conscious prioritization of correctness over safety and underscores the need for clearer signaling when UB is preserved rather than resolved.

Silent semantic patching is more dangerous than preserving UB Several translators silently alter semantics to avoid undefined behavior without generating runtime errors or strong indicators in the code. The evaluation suggests that this strategy is riskier than preserving UB or triggering runtime failures, as it can introduce latent, hard-to-detect behavioral changes.

Safety mechanisms shift failures from silent to observable modes Translators that rely on Rust's runtime checks tend to fail noisily rather than silently, which aligns better with modernization goals even when tests fail. The evaluation implicitly favors approaches that surface errors over those that mask them through silent correction.

6.4 HYBRIDIZATION, DIVERSITY, AND SYSTEM-LEVEL ADVANTAGE

Hybrid pipelines are already justified, not a future goal Empirical results show that no single technique simultaneously optimizes correctness, safety, and idiomaticity. Combining approaches now—particularly pairing test-driven equivalence checking with targeted unsafety elimination—offers immediate gains over monolithic pipelines.

Diversity of approaches yields complementary strengths No test case in the evaluation was failed by all performers, and most were failed by only one, demonstrating that different approaches succeed on different semantic corner cases. This strongly suggests that ensemble or hybrid translation strategies can outperform any single method in isolation.

6.5 SOURCE FIDELITY, CONFIGURABILITY, AND TOOLING ASSUMPTIONS

Preprocessing-induced loss of configurability is a systemic weakness Many translators implicitly collapse compile-time variability by translating only a single preprocessor configuration, silently discarding alternate code paths. Hayroll stands out as a concrete, reusable solution that preserves configurability and should be integrated more broadly across translation workflows.

Dependency and environment mismatch effects on builds Some of the performers build failures may be due to the packaging of dependencies. The translators are deployed in the performer's Docker containers and thus have the environment specified by the performer teams, but we analyze the translations as standalone Cargo projects on a separate system. In particular, as-yet-unreleased versions of c2rust-bitfields were leveraged by at least team Galois and required modifications to the produced Cargo.toml to build outside of the container to point to a repository explicitly.

This page intentionally left blank.

7. CONCLUSION

This report presented the first comprehensive test and evaluation of automated C-to-Rust translation systems under the DARPA TRACTOR program, using a common benchmark suite and a multidimensional evaluation framework encompassing functionality, safety, performance, and idiomaticity. Across six performer teams and two baselines, the results demonstrate that automated translation to Rust is already feasible for a nontrivial subset of C programs, with several approaches achieving high functional correctness on the initial battery while preserving competitive performance.

At the same time, the evaluation highlights that correctness beyond compilation remains the central challenge. Most failures arise from subtle semantic mismatches detectable only through test-based equivalence, underscoring the importance of rigorous behavioral validation and improved test coverage. The results further show that safety improvements are neither automatic nor uniform: performers make explicit and differing trade-offs between preserving C semantics, eliminating unsafe constructs, and surfacing undefined behavior through runtime checks or diagnostics.

A key finding of the evaluation is a broader tradeoff between correctness and idiomaticity. Static analysis-driven approaches generally achieved higher functional correctness, but often produced code that retained a more C-like structure and style. In contrast, LLM-driven approaches tended to generate more idiomatic, Rust-native code with substantially fewer Clippy lints and warnings, but with lower functional reliability. No single translation strategy dominates across all dimensions, suggesting that future progress will likely depend on hybrid pipelines that combine deterministic program analysis with LLM-guided transformation and validation.

The spectrum of different approaches, combined with the fact that no test case was failed by all performers, further suggests that hybrid and ensemble pipelines can outperform monolithic solutions. Tools such as Hayroll demonstrate the value of reusable infrastructure for addressing systemic challenges like conditional compilation, while emerging practices such as treating prompts as software artifacts point toward more disciplined and scalable use of LLMs.

Beyond demonstrating feasibility, these results underscore the necessity of a program like TRACTOR. While naïve baselines such as C2Rust and simple LLM prompting perform competitively on constrained benchmarks, the evaluation shows that they do not scale reliably to realistic workloads, deeper semantic corner cases, or security-relevant behaviors. The value of TRACTOR lies not in incremental gains on a single metric, but in enabling systematic, comparative progress across correctness, safety, performance, and maintainability through disciplined pipeline design and rigorous, multi-dimensional evaluation.

At the same time, the results make clear that substantial challenges remain, including robust handling of undefined behavior, preservation of compile-time configurability, stronger notions of equivalence beyond test coverage, and translation of large, security-critical codebases. These findings point both to meaningful progress toward scalable, security-driven software modernization and to the technical gaps that must be addressed to move beyond constrained settings. Future TRACTOR evaluations will expand benchmark scope, refine safety and equivalence metrics, and further explore integrated pipelines that combine deterministic analysis, LLM-guided transformation, and robust validation to advance the state of automated C-to-Rust translation.

This page intentionally left blank.

REFERENCES

[1] Office of the National Cyber Director, "Back to the building blocks: A path toward secure and measurable software," The White House, Technical rep. (2024), URL https://bidenwhitehouse.archives.gov/oncd/briefing-room/2024/02/26/memory-safety-fact-sheet/, technical report advocating adoption of memory safe programming languages.

[2] National Security Agency and Cybersecurity and Infrastructure Security Agency, "Memory safe languages: Reducing vulnerabilities in modern software development," NSA and CISA, Technical rep. (2025), URL https://www.cisa.gov/resources-tools/resources/memory-safe-languages-reducing-vulnerabilities-modern-software-development, joint cybersecurity information sheet highlighting benefits of memory safe languages.

[3] Immunant and Galois, "C2rust: A source-to-source transpiler for migrating c code to rust," Open-source tool, GitHub repository (2022), URL https://github.com/immunant/c2rust, c2Rust automatically translates C (C99) code into semantically equivalent Rust code, preserving functionality though often producing unsafe and unidiomatic Rust.

[4] TechRepublic, "Tiobe index for february 2026: Top 10 most popular programming languages," *TechRepublic* (2026), URL https://www.techrepublic.com/article/news-tiobe-index-language-rankings/, according to the TIOBE Programming Community Index, C and C++ consistently rank among the top programming languages worldwide.

[5] L. Szekeres, M. Payer, T. Wei, and D. Song, "Sok: Eternal war in memory," in *2013 IEEE Symposium on Security and Privacy (SP)*, IEEE (2013), pp. 48–62, URL https://ieeexplore.ieee.org/document/6547101/.

[6] H. Okhravi, "Memory safety," *IEEE Security & Privacy* 22(4), 13–15 (2024).

[7] E.A.O. Levy, "Smashing the stack for fun and profit," Phrack Magazine Issue 49 (1996), URL https://phrack.org/issues/49/14.html.

[8] Microsoft Corporation, "Trends, challenges, and strategic shifts in the software vulnerability mitigation landscape," Microsoft, Technical report (2019), URL https://github.com/microsoft/MSRC-SecurityResearch/blob/master/presentations/2019_02_BlueHatIL/2019_01%20-%20BlueHatIL%20-%20Trends%2C%20challenge%2C%20and%20shifts%20in%20software%20vulnerability%20mitigation.pdf, microsoft® presentation at BlueHat IL 2019 on software vulnerability trends.

[9] SOC Prime, "Cve-2026-20700: Apple patches zero-day exploited in sophisticated cyber attacks," SOC Prime blog post (2026), URL https://socprime.com/blog/cve-2026-20700-vulnerability/.

[10] Consumer Reports Advocacy, "Future of memory safety: Challenges and recommendations," Consumer Reports Advocacy report (2023), URL https://advocacy.consumerreports.org/wp-content/uploads/2023/11/ConsumerReportsonmemorysafety.pdf.

[11] Cybersecurity and Infrastructure Security Agency and Federal Bureau of Investigation, "Secure by design alert: Eliminating buffer overflow vulnerabilities," Threat advisory (2025), URL https://www.cisa.gov/resources-tools/resources/secure-design-alert-eliminating-buffer-overflow-vulnerabilities.

[12] A.A. Terekhov and C. Verhoef, "The realities of language conversions," *IEEE Software* 17(6), 111–124 (2000), URL https://doi.org/10.1109/52.895180, discusses practical challenges and limitations of automated language converters for legacy code.

[13] C. Cowan, M. Barringer, C. Beattie, G. Kroah-Hartman, M. Frantzen, and P. Lokier, "Stackguard: Automatic adaptive detection and prevention of buffer-overflow attacks," in *Proceedings of the 7th USENIX Security Symposium*, USENIX Association (1998), pp. 63–78, presents StackGuard, an early and influential stack canary based defense against stack buffer overflows.

[14] H. Shacham, M. Page, B. Pfaff, E.J. Goh, N. Modadugu, and D. Boneh, "On the effectiveness of address-space randomization," in *Proceedings of the 11th ACM Conference on Computer and Communications Security (CCS)*, ACM (2004), pp. 298–307.

[15] N. Burow, S.A. Carr, J. Nash, P. Larsen, M. Franz, S. Brunthaler, and M. Payer, "Control-flow integrity: Precision, security, and performance," *ACM Computing Surveys* 50(1), 16:1–16:33 (2017), survey of control-flow integrity mechanisms and their security and performance characteristics.

[16] H. Lefeuvre, N. Dautenhahn, D. Chisnall, and P. Olivier, "Sok: Software compartmentalization," in *Proceedings of the 46th IEEE Symposium on Security and Privacy (S&P'25)*, IEEE (2025), pp. 3107–3126.

[17] D. Song, J. Lettner, P. Rajasekaran, Y. Na, S. Volckaert, P. Larsen, and M. Franz, "Sok: Sanitizing for security," in *Proceedings of the 2019 IEEE Symposium on Security and Privacy*, IEEE (2019), pp. 1275–1295.

[18] A. Reitz, A. Fromherz, and J. Protzenko, "Starmalloc: Verifying a modern, hardened memory allocator," in *Proceedings of the 2024 ACM SIGPLAN Conference on Systems, Programming, Languages, and Applications: Software for Humanity (OOPSLA)* (2024), verified, concurrent, security-oriented memory allocator intended to harden against heap exploits and memory corruption attacks.

[19] MISRA, *MISRA C:2012 — Guidelines for the Use of the C Language in Critical Systems*, Motor Industry Software Reliability Association (2012), a comprehensive set of guidelines to improve safety, portability, and reliability of C code, widely adopted in safety-critical industries.

[20] R.C. Seacord, *The CERT® C Coding Standard: 98 Rules for Developing Safe, Reliable, and Secure Systems*, Addison-Wesley Professional (2014), a comprehensive secure coding standard for C that identifies common coding errors, including those that lead to memory corruption and other vulnerabilities.

[21] Google Project Zero, "Google project zero: Security research team focused on zero-day vulnerabilities," Online security research initiative (2014), URL https://projectzero.google/, project Zero is a Google security research team dedicated to finding and publicly reporting zero-day vulnerabilities in software and hardware systems to improve overall security.

[22] R.N. Watson, J. Baldwin, D. Chisnall, T. Chen, J. Clarke, B. Davis, N. Filardo, B. Gutstein, G. Jenkinson, B. Laurie, et al., "It is time to standardize principles and practices for software memory safety," *Communications of the ACM* (2025).

[23] S. Klabnik and C. Nichols, *The Rust Programming Language*, No Starch Press (2019), URL https://doc.rust-lang.org/book/, 2nd edition.

[24] M. Emre, R. Schroeder, K. Dewey, and B. Hardekopf, "Translating c to safer rust," *Proc. ACM Program. Lang.* 5(OOPSLA), 121 (2021).

[25] M. Ling, Y. Yu, H. Wu, Y. Wang, and J.R. Cordy, "In rust we trust: A transpiler from unsafe c to safer rust," (2022), available at research.cs.queensu.ca.

[26] A. Fromherz and J. Protzenko, "Compiling c to safe rust, formalized," (2024).

[27] R. Li, B. Wang, T. Li, P. Saxena, and A. Kundu, "Translating c to rust: Lessons from a user study," in *NDSS Symposium 2025* (2025).

[28] M. Farrukh, S. Shah, B. Coskun, and M. Polychronakis, "Safetrans: Llm-assisted transpilation from c to rust," (2025).

[29] Y. Gao, C. Wang, P. Huang, X. Liu, M. Zheng, and X. Zhang, "Pr2: Peephole raw pointer rewriting with llms for translating c to safer rust," (2025).

[30] T. Zhou, Z. Zhang, H. Lin, S. Jha, M. Christodorescu, K. Levchenko, and V. Chandrasekaran, "Sactor: Llm-driven correct and idiomatic c to rust translation with static analysis and ffi-based verification," (2025), URL https://arxiv.org/abs/2503.12511.

[31] S. Klabnik and C. Nichols, *The Rust Programming Language*, No Starch Press (2018), URL https://doc.rust-lang.org/book/.

[32] Defense Advanced Research Projects Agency, "Tractor: Translating all c to rust program," Program overview on DARPA.mil (2024), URL https://www.darpa.mil/research/programs/translating-all-c-to-rust, describes the TRACTOR initiative aimed at automating the translation of legacy C code to Rust to eliminate memory safety vulnerabilities.

[33] O. Khattab, A. Singhvi, P. Maheshwari, Z. Zhang, K. Santhanam, S. Vardhamanan, S. Haq, A. Sharma, T.T. Joshi, H. Moazam, et al., "Dspy: Compiling declarative language model calls into self-improving pipelines," *arXiv preprint arXiv:2310.03714* (2023).

[34] L.A. Agrawal, S. Tan, D. Soylu, N. Ziems, R. Khare, K. Opsahl-Ong, A. Singhvi, H. Shandilya, M.J. Ryan, M. Jiang, C. Potts, K. Sen, A.G. Dimakis, I. Stoica, D. Klein, M. Zaharia, and O. Khattab, "Gepa: Reflective prompt evolution can outperform reinforcement learning," (2025), URL https://arxiv.org/abs/2507.19457.

[35] B. Pappas and P. Gazzillo, "Semantic analysis of macro usage for portability," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, New York, NY, USA: Association for Computing Machinery (2024), ICSE '24, URL https://doi.org/10.1145/3597503.3623323.

[36] M. Shetty, N. Jain, A. Godbole, S.A. Seshia, and K. Sen, "Syzygy: Dual code-test c to (safe) rust translation using llms and dynamic analysis," (2024), URL https://arxiv.org/abs/2412.14234.

[37] T. Zhou, Z. Zhang, H. Lin, S. Jha, M. Christodorescu, K. Levchenko, and V. Chandrasekaran, "Sactor: Llm-driven correct and idiomatic c to rust translation with static analysis and ffi-based verification," (2025), URL https://arxiv.org/abs/2503.12511.

[38] H. Zhang, C. David, Y. Yu, and M. Wang, "Ownership guided c to rust translation," in *Computer Aided Verification: 35th International Conference, CAV 2023, Paris, France, July 17–22, 2023, Proceedings, Part III*, Berlin, Heidelberg: Springer-Verlag (2023), pp. 459–482, URL https://doi.org/10.1007/978-3-031-37709-9_22.

[39] E.S. Hald, "Educational undefined behavior technical report," URL https://www.open-std.org/JTC1/sc22/wg14/www/docs/n3308.pdf.

[40] R. Jung, B. Kimock, C. Poveda, E.S. Muñoz, O. Scherer, and Q. Wang, "Miri: Practical undefined behavior detection for rust," *Proceedings of the ACM on Programming Languages* 10(POPL), 1383–1411 (2026).

[41] "perf-stat(1)—linux manual page," URL https://man7.org/linux/man-pages/man1/perf-stat.1.html.

[42] N. Nethercote and J. Seward, "Valgrind: a framework for heavyweight dynamic binary instrumentation," in *Proceedings of the 28th ACM SIGPLAN Conference on Programming Language Design and Implementation*, New York, NY, USA: Association for Computing Machinery (2007), PLDI '07, p. 89–100, URL https://doi.org/10.1145/1250734.1250746.

[43] "Clippy documentation," URL https://doc.rust-lang.org/stable/clippy/index.html.

[44] T. Yin and contributors, "Lizard: A simple code complexity analyser," https://github.com/terryyin/lizard.

[45] S. Barrett, "stb_perlin: Public domain perlin noise implementation," https://github.com/nothings/stb (2017), single-file public domain library, stb_perlin.h.

[46] D.J. Bernstein, A. Hülsing, S. Kölbl, R. Niederhagen, J. Rijneveld, and P. Schwabe, "The sphincs+ signature framework," in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, New York, NY, USA: Association for Computing Machinery (2019), CCS '19, p. 2129–2146, URL https://doi.org/10.1145/3319535.3363229.

[47] E. Ronen, "splincsplusc: Sphincs+c," https://github.com/eyalr0/sphincsplusc (2022).

[48] J.P. Aumasson, L. Henzen, W. Meier, and R.C.W. Phan, "Sha-3 proposal blake," Submission to NIST (2008), URL http://ehash.iaik.tugraz.at/uploads/0/06/Blake.pdf.

A AI TERMINOLOGY GLOSSARY

In this section, we provide a brief description of AI-related terms used in this report. Interested readers should refer to AI-related literature for a more complete and deeper treatment of the subject.

LLM (Large Language Model) A neural network model trained on large text corpora that can generate, transform, or reason about code and natural language.

Prompt A structured natural-language instruction passed to an LLM to specify a translation task, constraints, and expected output format.

Prompt Engineering The practice of deliberately designing, structuring, and tuning prompts to guide LLM behavior toward a desired output.

Prompt Template A reusable, role-specific prompt skeleton parameterized for different pipeline stages (e.g., unsafe removal, compile repair, test repair).

Prompt Contract / Output Contract A strict specification embedded in a prompt that constrains the format of the LLM's response (e.g., JSON-only, fenced code blocks) to enable deterministic downstream parsing.

Prompt-as-Code Treating prompts as versioned, structured software artifacts with typed fields and optimization targets, rather than ad hoc instructions.

Prompt Optimization Automatically tuning prompt text against a measurable metric (e.g., "compiles cleanly") rather than relying on human authorship alone.

DSPy A declarative framework that abstracts prompt engineering into typed "signatures" and compiles them into self-improving LLM pipelines optimizable against user-defined metrics.

GEPA (Genetic-Pareto Style Optimizer) An evolutionary multi-objective optimizer that maintains a population of candidate prompts and iteratively improves them via a reward function to find globally optimal prompts.

Context Pressure The strain on an LLM caused by providing too much input at once (e.g., an entire codebase), increasing the risk of errors, missed details, and loss of error provenance.

Whole-Project Prompting Sending the entire codebase as context in a single LLM call to maximize global coherence, at the cost of increased context pressure.

Snippet-Level / Symbol-Level Translation An LLM prompting strategy that translates one function or symbol at a time in dependency order, reducing context size per call and localizing failures.

Dependency-Ordered Translation Processing code units in the order of their dependencies so each LLM call can use already-translated code as structured context.

One-Shot Conversion Translating code in a single LLM call without iteration or feedback, contrasted with multi-stage repair pipelines.

Repair Loop / Build-Driven Repair An iterative process where compiler errors or test failures are fed back into the LLM as structured feedback to progressively fix a translation.

Feedback Loop The mechanism by which compiler output, test results, or discrepancy reports are injected back into the LLM prompt to guide the next rewrite iteration.

RAG-Like Context Selection A retrieval-augmented-generation-inspired strategy for selecting only the most relevant code slices as LLM context rather than sending everything at once.

Stochastic Rewrite A non-deterministic LLM-based code transformation whose output may vary across runs, requiring provenance tracking and bounded iteration to manage.

Orchestration The coordination layer that schedules, sequences, and manages multiple LLM calls and analysis tools within a translation pipeline.

Semantic Guardrails Prompt-level constraints (e.g., "implement bitwise exactly," "align integer widths") that prevent the LLM from introducing subtle semantic drift during translation.

Hard Safety Guardrails Build-breaking enforcement rules that cause compilation to fail if the LLM produces disallowed constructs (e.g., any use of `unsafe`), making safety non-negotiable.

Discrepancy Report A structured, LLM-generated report identifying behavioral differences between a C source file and its Rust translation, used as targeted context for a subsequent repair pass.

Silent Semantic Patching An LLM silently altering program semantics (e.g., removing undefined behavior) without flagging the change, introducing risk because the output diverges from the original without a visible signal.

Translation Granularity The scope of code processed per LLM call, a primary architectural design axis ranging from whole-project (high coherence, high context pressure) to symbol-level (low context pressure, high orchestration complexity).

LLM-Forward Pipeline An architecture where the LLM is the primary driver of translation from the outset, as opposed to a transpiler-first approach that uses LLMs only for refinement.

Test-Based Equivalence The practical correctness criterion that a translation is "correct" if it passes the same test vectors as the original, as opposed to formal semantic equivalence.

Semantic Risk Model Each team's implicit encoding of which semantic deviations from the original C are acceptable (e.g., whether to preserve undefined behavior or silently fix it).

Provenance Tracking / Provenance Graph Recording each LLM transformation as a node in a graph so that the origin of every code artifact is traceable, supporting auditability and rollback.

MVIR (Multi-Version Intermediate Representation) Galois's provenance-tracking abstraction that records each transpilation attempt, LLM rewrite, and repair as a versioned node, enabling the pipeline to prefer or revert to any prior candidate.

This page intentionally left blank.

This page intentionally left blank.